Harmony - simulator design and use

Azadeh Kermansaravi, Debottam Mukherjee, Haixiao Li, Saif Alsarayreh, Robert Dimitrovski, Aleksandra Lekić

September 2024

Contents

1 Introduction

2 AC/MTDC Optimal Power Flow

2.1 Getting Start

This section provides a concise guide on running simulations, modifying input data, and accessing results. It serves as a basic introduction to help to get started with the code.

2.1.1 Pacakge Reuqirements

MATLAB code has the requirements: ✓MATPOWER version 7.0 or later. ✓YALMIP version 2017 or later. JULIA code has the requirements: ✓JUMP version 0.21 or later.

2.1.2 Running A Simulation

The primary functionality of `acmtdcpf.m`/`acmtdcpf.jl` is to solve the AC/DC optimal power flow (OPF) problem. Running a simulation using `acmtdcpf.m`/`acmtdcpf.jl` requires: (1) preparing the AC system/VSC-MTDC system input date; (2) specifying the interconnection between the AC system and MTDC system; (3) invoking the function to run the simulation; (4) accessing and viewing the results.

The classic AC power grid input data is represented by the MATPOWER-CASE struct, denoted by `mpc`. Following this structure, in the MATLAB code, an extended MATPOWER-CASE is required to store information about the nodes, branches, and generators of multiple AC grids. Similarly, an extended MATPOWER-CASE is needed to store information about the nodes and branches of the MTDC grid, as well as the nodes, branches, and interconnection of the VSC stations. In JULIA, the input data for the AC grids and the MTDC grid are stored in dictionaries using the functions `create_ac.jl` and `create_dc.jl`, which are quite similar to the MATPOWER-CASE struct. The following syntax is used to run AC/DC OPF.

```
% 'ac5ac9' 5-node ac grid and 9-node ac grid
% 'mtdc3slack_a' 3-node mtdc grid
acmtdcpf('ac5ac9', 'mtdc3slack_a')
```

2.1.3 Accessing the Results

By default, the results of the optimization running are printed on the screen, following the traditional MATPOWER display style. The AC and MTDC node information are selected to display.

2.2 Input Data

This section introduces the input format. The input data contains two main parts: AC system and VSC-MTDC system. This chapter introduces the detailed data matrices in AC AND VSC-MTDC systems (the data useful for OPF is mainly listed).

2.2.1 AC System

In MATALB version, the AC system input data format is stored in the struct data `mpc`, which is basically the same as MATPOWER. Note that the data related to multiple AC grids can be stored in one struct `mpc`. Different AC grids are distinguished from each other through the flag `grid`. Table 1 lists the critical parameters that need to be defined in the struct `mpc`. If in JULIA version, the AC system input data is converted to a dictionary data `ac`, which is listed in Table 2.

2.2.2 VSC-MTDC System

In MATALB version, the VSC-MTDC system input data is stored in struct data `mpc` and contains three matrices, that are `mpc.bus`, `mpc.branch`, and `mpc.conv`. `mpc.bus` and `mpc.branch` are used to store the MTDC bus and branch data, respectively. `mpc.conv` is used to store the VSC data, including bus, branch,

Name	Description
`mpc.baseMVA`	Base power capacity [MVA]
`mpc.bus(:, bus_i)`	Numbers of buses
`mpc.bus(:, Pd)`	Active power loads [p.u.]
`mpc.bus(:, Qd)`	Reactive power loads [p.u.]
`mpc.bus(:, Vmax)`	Upper bounds of voltage magnitudes [p.u.]
`mpc.bus(:, Vmin)`	Lower bounds of voltage magnitudes [p.u.]
`mpc.bus(:, grid)`	Grid numbers that AC buses correspond to
`mpc.gen(:, Vg)`	Voltage references of generators [p.u.]
`mpc.gen(:, Pmax)`	Upper bounds of generator active power outputs [MW]
`mpc.gen(:, Pmin)`	Lower bounds of generator active power outputs [MW]
`mpc.gen(:, Qmax)`	Upper bounds of generator reactive power outputs [MVAr]
`mpc.gen(:, Qmin)`	Lower bound of generator reactive power outputs [MVAr]
`mpc.gen(:, grid)`	Grid numbers that generator correspond to [-]
`mpc.branch(:, fbus)`	From buses of branches
`mpc.branch(:, tbus)`	To buses of branches
`mpc.branch(:, r)`	Resistances of branches [p.u.]
`mpc.branch(:, x)`	Reactances of branches [p.u.]
`mpc.branch(:, b)`	Shut susceptances of branches [p.u.]
`mpc.branch(:, grid)`	Grid numbers that branch correspond to
`mpc.gencost(:, n)`	Orders of generator coefficients
`mpc.gencost(:, c2)`	Quadratic generation cost coefficients
`mpc.gencost(:, c1)`	Linear generation cost coefficients
`mpc.gencost(:, c0)`	Constant generation cost coefficients
`mpc.gencost(:, grid)`	Grid numbers of generator coefficients

Table 1: AC System Parameters for OPF (MATLAB)

operation mode, etc.. Table 3 lists the critical parameters that need to be defined in struct `mpc`. If in JULIA version, the VSC-MTDC system input data is converted to a dictionary data `dc`, which is listed in Table 4.

2.3 Problem Modeling

This section introduces the modeling of the AC/MTDC power system. The necessary constraint and objective functions to describe the optimal steady-state behavior for the AC/MTDC power system are introduced. Symbols appearing in subsequent formulas can be found below.

Indexes and Sets

$(\bar{\cdot})/(\underline{\cdot})$ -The upper/lower bound of the corresponding optimization variables and parameters

$|\cdot|$ -Magnitude

$\mathcal{L}$ -Branch set

$\mathcal{N}$ -Node set

AC -Alternating current

cost -Generation cost

droop -Droop parameters

gen -Generator

Name	Description
`ac["baseMVA"]`	Base power capacity [MVA]
`ac["bus"][:, Pd]`	Active power loads [MW]
`ac["bus"][:, Qd]`	Reactive power loads [MW]
`ac["bus"][:, Vmax]`	Upper bounds of voltage magnitudes [p.u.]
`ac["bus"][:, Vmin]`	Lower bounds of voltage magnitudes [p.u.]
`ac["bus"][:, grid]`	Grid numbers that AC buses correspond to
`ac["gen"][:, Vg]`	Voltage references of generators [p.u.]
`ac["gen"][:, Pmax]`	Upper bounds of generator active power outputs [MW]
`ac["gen"][:, Pmin]`	Lower bounds of generator active power outputs [MW]
`ac["gen"][:, Qmax]`	Upper bounds of generator reactive power outputs [MVAr]
`ac["gen"][:, Qmin]`	Lower bounds of generator reactive power outputs [MVAr]
`ac["gen"][:, grid]`	Grid numbers that generator correspond to
`ac["branch"][:, fbus]`	From buses of branches
`ac["branch"][:, tbus]`	To buses of branches
`ac["branch"][:, r]`	Resistances of branches [p.u.]
`ac["branch"][:, x]`	Reactances of branches [p.u.]
`ac["branch"][:, b]`	Shut susceptances of branches [p.u.]
`ac["branch"][:, grid]`	Grid numbers that branch correspond to
`ac["gencost"][:, n]`	Orders of generator coefficients
`ac["gencost"][:, c2]`	Quadratic generation cost coefficients
`ac["gencost"][:, c1]`	Linear generation cost coefficients
`ac["gencost"][:, c0]`	Constant generation cost coefficients
`ac["gencost"][:, grid]`	Grid numbers of generator coefficients

Table 2: AC System Parameters (Julia)

h, i, j -System node

ij -System branch

load -Load

loss -Power loss

MTDC -Multi-terminal direct current

ref -Reference value

VSC -Voltage source converter

Variables and Parameters Related to the AC Grid

θ_{ij}^{AC} -Phase difference for AC branch ij

$c_{i,2}^{AC}/c_{i,1}^{AC}/c_{i,0}^{AC}$ -Quadratic/Linear/Constant Generation coefficient at AC bus i.

p_{ij}^{AC}/q_{ij}^{AC} -Real/Reactive power for AC branch ij

$p_{i,A2V}^{AC}/q_{i,A2V}^{AC}$ -Real/Reactive power delivered from the AC grid to the VSC station at AC bus i

$p_{i,gen}^{AC}/q_{i,gen}^{AC}$ -Real/Reactive power outputs of generator at AC bus i

$p_{i,load}^{AC}/q_{i,load}^{AC}$ -Real/Reactive power loads at AC bus i

p_{i}^{AC}/q_{i}^{AC} -Real/Reactive power injection at AC bus i

Name	Description
`mpc.baseMW`	Base power capacity [MW]
`mpc.pol`	1→monopolar MTDC grid; 2→bipolar
`mpc.bus(:, bus_i)`	Numbers of buses
`mpc.bus(:, Vmax)`	Upper bounds of voltage magnitudes at MTDC buses [p.u.]
`mpc.bus(:, Vmin)`	Lower bounds of voltage magnitudes at MTDC buses [p.u.]
`mpc.conv(:, busdc_i)`	MTDC buses that VSCs are connected to
`mpc.conv(:, busac_i)`	AC buses that VSCs are connected to
`mpc.conv(:, grid)`	AC grids to which the AC buses connected to VSCs correspond
`mpc.conv(:, type_dc)`	Control modes of VSCs DC side(1→DC power control; 2→DC voltage control; 3→DC droop control)
`mpc.conv(:, type_ac)`	Control modes of VSCs AC side(1→AC voltage control; 2→Reactive power control)
`mpc.conv(:, rtf)`	Transformers resistances [p.u.]
`mpc.conv(:, xtf)`	Transformers reactances [p.u.]
`mpc.conv(:, rc)`	Phase reactors resistances [p.u.]
`mpc.conv(:, xc)`	Phase reactors reactances [p.u.]
`mpc.conv(:, Vmax)`	Upper bounds of voltage magnitudes of VSCs AC side [p.u.]
`mpc.conv(:, Vmin)`	Lower bounds of voltage magnitudes of VSCs AC side [p.u.]
`mpc.conv(:, Imax)`	Maximums of VSC phase current magnitudes [p.u.]
`mpc.conv(:, LossA)`	Constant coefficients of VSC power losses [p.u.]
`mpc.conv(:, LossB)`	Linear coefficients of VSC power losses [p.u.]
`mpc.conv(:, LossCrec)`	Quadratic coefficients of VSC power losses for the rectifier mode [p.u.]
`mpc.conv(:, LossCinv)`	Quadratic coefficients of VSC power losses for the inverter mode [p.u.]
`mpc.conv(:,droop)`	Slopes in DC voltage droop control
`mpc.conv(:,Pdcset)`	DC voltage droop power set points [MW]
`mpc.conv(:,Vdcset)`	DC voltage droop voltage set points [p.u.]

Table 3: VSC-MTDC System Parameters (MATLAB)

v_i^{AC} -AC bus i voltage

Variables and Parameters Related to the MTDC Grid

$k_{i,droop}^{MTDC}$ -Droop slope at MTDC bus i

$p_{i,ref}^{MTDC}$ -Power reference of DC power control/DC droop control at MTDC bus i

p_{jh}^{MTDC} -Power of MTDC branch jh

v_j^{MTDC} -MTDC bus j voltage

$V_{i,ref}^{MTDC}$ -Voltage reference of DC voltage control/DC droop control at MTDC bus i

Variables and Parameters Related to the VSC Station

$\theta_s^{VSC}/\theta_f^{VSC}/\theta_c^{VSC}$ -VSC bus $s/f/c$ phase

$a_{loss}^{VSC}/b_{loss}^{VSC}/c_{loss}^{VSC}$ -Constant/Linear/Quadratic coefficient of VSC cpower loss function

b_f^{VSC} -Susceptance for VSC bus f to ground

$b_{sf}^{VSC}/b_{fc}^{VSC}$ -Susceptance for VSC branch sf/fc

Name	Description
`dc["baseMW"]`	Base power capacity [MW]
`dc["pol"]`	1→monopolar MTDC grid; 2→bipolar
`dc["bus"][:, bus_i]`	Numbers of buses
`dc["bus"][:, Vmax]`	Upper bounds of voltage magnitudes at MTDC buses [p.u.]
`dc["bus"][:, Vmin]`	Lower bounds of voltage magnitudes at MTDC buses [p.u.]
`dc["conv"][:, busdc_i]`	MTDC buses that VSCs are connected to
`dc["conv"][:, busac_i]`	AC buses that VSCs are connected to
`dc["conv"][:, grid]`	AC grids to which the AC buses connected to VSCs correspond
`dc["conv"][:, type_dc]`	Control modes of VSCs DC side (1→DC power control; 2→DC voltage control; 3→DC droop control)
`dc["conv"][:, type_ac]`	Control modes of VSCs AC side (1→AC voltage control; 2→Reactive power control)
`dc["conv"][:, rtf]`	Transformer resistances [p.u.]
`dc["conv"][:, xtf]`	Transformer reactances [p.u.]
`dc["conv"][:, rc]`	Phase reactor resistances [p.u.]
`dc["conv"][:, xc]`	Phase reactor reactances [p.u.]
`dc["conv"][:, Vmax]`	Upper bounds of voltage magnitudes of VSCs AC side [p.u.]
`dc["conv"][:, Vmin]`	Lower bounds of voltage magnitude at VSCs AC side [p.u.]
`dc["conv"][:, Imax]`	Maximums of VSC phase current magnitudes [p.u.]
`dc["conv"][:, LossA]`	Constant coefficients of VSC power losses
`dc["conv"][:, LossB]`	Linear coefficients of VSC power losses
`dc["conv"][:, LossCrec]`	Quadratic coefficients of VSC power losses for the rectifier mode
`dc["conv"][:, LossCinv]`	Quadratic coefficients of VSC power losses for the inverter mode
`dc["conv"][:, droop]`	Slopes in DC voltage droop control
`dc["conv"][:, Pdcset]`	DC voltage droop power set points [MW]
`dc["conv"][:, Vdcset]`	DC voltage droop voltage set points [p.u.]

Table 4: VSC-MTDC System Parameters (Julia)

$g_{sf}^{VSC}/g_{fc}^{VSC}$ -Conductance for VSC branch sf/fc

I_c^{VSC} -Phase current at VSC bus c

p_{loss}^{VSC} -VSC power loss

$q_{s,ref}^{VSC}$ -Power reference of reactive power control at VSC bus s

$v_s^{VSC}/v_f^{VSC}/v_c^{VSC}$ -VSC bus $s/f/c$ voltage

$v_{s,ref}^{VSC}$ -Voltage reference of AC voltage control at VSC bus s

2.3.1 AC Gird Modeling

The active and reactive power flow equations on a branch are given by (2.1) and (2.2), respectively:

$$p_{ij}^{AC} = g_{ij}^{AC} \left|v_i^{AC}\right|^2 - \left|v_i^{AC}\right| \left|v_j^{AC}\right| \left(g_{ij}^{AC} \cos\theta_{ij}^{AC} + b_{ij}^{AC} \sin\theta_{ij}^{AC}\right), \quad \forall\, i,j \in \mathcal{N}^{AC}, \quad \forall\, (i,j) \in \mathcal{L}^{AC} \tag{2.1}$$

$$q_{ij}^{AC} = -b_{ij}^{AC} \left|v_i^{AC}\right|^2 + \left|v_i^{AC}\right| \left|v_j^{AC}\right| \left(b_{ij}^{AC} \cos\theta_{ij}^{AC} - g_{ij}^{AC} \sin\theta_{ij}^{AC}\right), \quad \forall\, i,j \in \mathcal{N}^{AC}, \quad \forall\, (i,j) \in \mathcal{L}^{AC}. \tag{2.2}$$

The active and reactive nodal power injections can be further expressed as (2.3):

$$p_i^{AC} = \sum_{(i,j)} p_{ij}^{AC} + \left(\sum_j G_{ij}^{AC}\right) (v_i^{AC})^2, \quad q_i^{AC} = \sum_{(i,j)} q_{ij}^{AC} + \left(\sum_j -B_{ij}^{AC}\right) (v_i^{AC})^2. \tag{2.3}$$

Regarding p_i^{AC} and q_i^{AC}, they can be explicitly expressed as (2.4):

$$p_i^{AC} = p_{i,gen}^{AC} - p_{i,load}^{AC} - p_{i,A2V}^{AC}, \quad q_i^{AC} = q_{i,gen}^{AC} - q_{i,load}^{AC} - q_{i,A2V}^{AC}, \quad i \in \mathcal{N}^{AC}. \tag{2.4}$$

.

The necessary operational constraints include the generator power output constraint (2.5), branch apparent power constraint (2.6), and nodal voltage limit constraint (2.7), such that:

$$\underline{p}_{i,gen}^{AC} \le p_{i,gen}^{AC} \le \overline{p}_{i,gen}^{AC}, \quad \underline{q}_{i,gen}^{AC} \le q_{i,gen}^{AC} \le \overline{q}_{i,gen}^{AC}, \quad i \in \mathcal{N}^{AC} \tag{2.5}$$

$$(p_{ij}^{AC})^2 + (p_{ij}^{AC})^2 \le (\bar{s}_{ij}^{AC})^2, \quad (i,j) \in \mathcal{L}^{AC} \tag{2.6}$$

$$\underline{\left|v_i^{AC}\right|} \le \left|v_i^{AC}\right| \le \overline{\left|v_i^{AC}\right|}, \quad i \in \mathcal{N}^{AC}. \tag{2.7}$$

2.3.2 MTDC Grid Modeling

The MTDC branch power flow is expressed as (2.8):

$$p_{jh}^{MTDC} = \rho^{MTDC} \cdot v_j^{MTDC}\left(v_j^{MTDC} - v_h^{MTDC}\right) y_{jh}^{MTDC}, \quad j,h \in \mathcal{N}^{MTDC}, \quad (j,h) \in \mathcal{L}^{MTDC} \tag{2.8}$$

The MTDC nodal power injection can be further expressed as (2.9):

$$p_j^{MTDC} = \sum_{(j,h)} p_{jh}^{MTDC}. \tag{2.9}$$

The necessary operational constraints include the nodal voltage limit constraint (2.10), such that:

$$\underline{v}_i^{MTDC} \le v_i^{MTDC} \le \overline{v}_i^{MTDC}, \quad i \in \mathcal{N}^{MTDC} \tag{2.10}$$

VSC has three control mode for DC side, which are DC voltage control (2.11), DC power control (2.12), and DC droop control (2.13), and additional constraints related to DC node need to be considered:

$$p_i^{MTDC} = p_{i,ref}^{MTDC}, \tag{2.11}$$

$$v_i^{MTDC} = v_{i,ref}^{MTDC}, \tag{2.12}$$

$$p_i^{MTDC} = p_{i,ref}^{MTDC} - \frac{1}{k_{i,droop}^{MTDC}}\left(v_i^{MTDC} - v_{i,ref}^{MTDC}\right). \tag{2.13}$$

2.3.3 VSC Station Modeling

The power flow model of the VSC station is depicted in Figure 2.1, which is from the AC point of common coupling (PCC) to the DC terminal. The power injections at the PCC (node s) are expressed as (2.14) and (2.15):

$$p_s^{VSC} = -\left|v_s^{VSC}\right|^2 g_{sf}^{VSC} + \left|v_s^{VSC}\right|\left|v_f^{VSC}\right|\left[g_{sf}^{VSC}\cos\left(\theta_s^{VSC} - \theta_f^{VSC}\right) + b_{sf}^{VSC}\sin\left(\theta_s^{VSC} - \theta_f^{VSC}\right)\right], \tag{2.14}$$

$$q_s^{VSC} = \left|v_s^{VSC}\right|^2 b_{sf}^{VSC} + \left|v_s^{VSC}\right|\left|v_f^{VSC}\right|\left[g_{sf}^{VSC}\cos\left(\theta_s^{VSC} - \theta_f^{VSC}\right) - b_{sf}^{VSC}\sin\left(\theta_s^{VSC} - \theta_f^{VSC}\right)\right]. \tag{2.15}$$

The power injections at the AC terminal (node c) are expressed as (2.16) and (2.17):

$$p_c^{VSC} = \left|v_c^{VSC}\right|^2 g_{fc}^{VSC} + \left|v_f^{VSC}\right|\left|v_c^{VSC}\right|\left[g_{fc}^{VSC}\cos\left(\theta_f^{VSC} - \theta_c^{VSC}\right) - b_{fc}^{VSC}\sin\left(\theta_f^{VSC} - \theta_c^{VSC}\right)\right], \tag{2.16}$$

$$q_c^{VSC} = -\left|v_c^{VSC}\right|^2 b_{fc}^{VSC} + \left|v_f^{VSC}\right|\left|v_c^{VSC}\right|\left[g_{fc}^{VSC}\sin\left(\theta_f^{VSC} - \theta_c^{VSC}\right) + b_{fc}^{VSC}\cos\left(\theta_f^{VSC} - \theta_c^{VSC}\right)\right]. \tag{2.17}$$

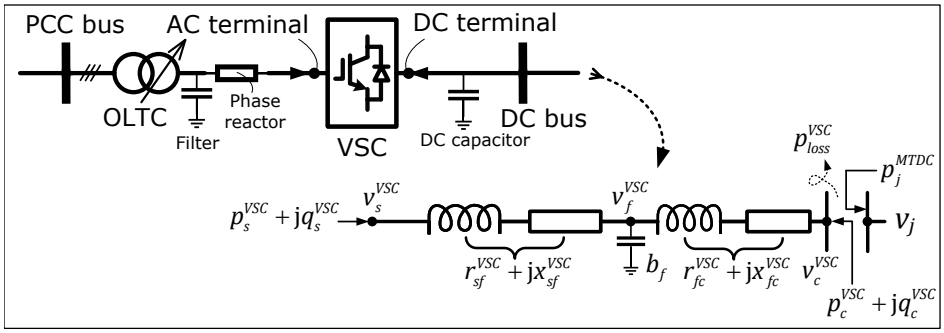

Figure 2.1: Equivalent power flow model of the VSC station.

The reactive power at the filter (node f) node is given by (2.18):

$$q_f^{VSC} = -\left|v_f^{VSC}\right|^2 b_f^{VSC}. \tag{2.18}$$

If we consider the latest converter with the multilevel converter topologies, the low-pass AC filter is not required and $b_f^{VSC} = 0$.

The power coupling between the AC terminal and DC terminal is expressed as (2.19):

$$p_c^{VSC} + p_{loss}^{VSC} + p_j^{DC} = 0, \quad j \in \mathcal{N}^{MTDC}. \tag{2.19}$$

The converter loss p_{loss}^{VSC} is formulated as a quadratic equation (2.20) with respect to the converter current magnitude $|I_c|$, such that:

$$p_{\text{loss}}^{VSC} = a_{loss}^{VSC} + b_{loss}^{\text{VSC}} \left|I_c^{VSC}\right| + c_{loss}^{VSC} \left|I_c^{VSC}\right|^2, \tag{2.20}$$

with

$$\left|I_c^{VSC}\right| = \sqrt{\frac{\left(p_c^{VSC}\right)^2 + \left(q_c^{VSC}\right)^2}{\left(v_c^{VSC}\right)^2}}. \tag{2.21}$$

VSC has two control mode for AC side, which are AC voltage control (2.22) and reactive power control (2.23), and additional constraints related to PCC need to be considered:

$$v_s^{VSC} = v_{s,ref}^{VSC}, \tag{2.22}$$

$$q_s^{VSC} = q_{s,ref}^{VSC}. \tag{2.23}$$

2.3.4 Optimization Objectives

$$\min \sum_i c_{i,2}^{AC} \left(P_{i,G}^{AC}\right)^2 + c_{i,1}^{AC} P_{i,G}^{AC} + c_{i,0}^{AC} \quad i \in \mathcal{N}^{AC}, \tag{2.24}$$

where optimzation objective of AC/MTDC grid formulated by (2.24) is to minimize the total generation costs.

2.4 Examples

We analyze an interconnected system formed by a power system of five buses and nine buses, an MTDC grid of three terminals.

For MATLAB code, we can directly input the running command like:

```
clc;
clear all;
acmtdcpf('ac5ac9','mtdc3slack_a');
```

For JULIA code, before input the running command, some additional `function.jl` need to be manually added like:

```
using JuMP, Ipopt, Printf
include("create_ac.jl") # define interconnceted ac grids
include("create_dc.jl") # define multi-terminal dc grid
include("makeYbus.jl") # calculate Bus Admittance Matrix
include("params_dc.jl") # obtain ac grid parameters
include("params_ac.jl") # obtain dc grid parameters
include("acmtdcpf.jl") # run ac/dc opf
acmtdcpf("ac5ac9","mtdc3slack_a")
```

By default, the `output results` are printed and looks like:

```
================================================================================
|   AC  Bus Data                                                               |
================================================================================
 Bus      Area      Voltage          Generation             Load
 #        #    Mag [pu] Ang [deg]   P [MW]   Q [MVAr]   P [MW]  Q [MVAr]
-----    -----  --------  --------  --------  --------  -------- --------
   1        1    1.060     0.000*   191.77     28.72      0.00     0.00
   2        1    1.016    -4.364       -         -       20.00    10.00
   3        1    1.027    -5.501       -         -       45.00    15.00
   4        1    1.032    -5.713     40.00     40.97     40.00     5.00
   5        1    1.000    -7.422       -         -       60.00    10.00
   1        2    1.040     0.000*    70.70     19.04      0.00     0.00
   2        2    1.038     4.007    111.36      8.82      0.00     0.00
   3        2    1.000    -1.318       -         -        0.00     0.00
   4        2    1.030    -2.178       -         -        0.00     0.00
   5        2    1.022    -2.419     79.79     -2.31     90.00    30.00
   6        2    1.018    -1.992       -         -        0.00     0.00
   7        2    1.015    -2.912       -         -      100.00    35.00
   8        2    1.035     0.293       -         -        0.00     0.00
   9        2    1.004    -5.067       -         -      125.00    50.00
-----    ----- --------  --------  --------  --------  -------- --------
 The total generation cost is USD 9973.80/MWh(EUR 9235.00/MWh)


================================================================================
|     DC bus data                                                              |
================================================================================
 Bus   Bus    AC   DC Voltage   DC Power   PCC Bus Injection   Converter loss
 DC #  AC #  Area   Vdc [pu]    Pdc [MW]   Ps [MW]  Qs [MVAr]  Conv_Ploss [MW]
-----  ----  ----  ---------    --------   -------  --------    --------
   1     2     1     1.008       58.512    60.000    40.000       1.407
   2     3     2     1.000      -21.721   -20.452    30.547       1.247
   3     8     2     0.998      -36.252   -35.000    -5.000       1.234
-----  ----  ----  ---------    --------   -------  --------    --------
 The total converter losses is 3.888 MW

 Power flow computation time is 0.852s
```

3 Mathematical Framework Implementation

3.1 Getting Start

The increasing complexity of modern power systems, especially Multi-terminal Voltage Direct Current (MT-HVDC) systems, requires an advanced simulation tool capable of addressing unique challenges posed by such systems. This project aims to implement a comprehensive mathematical framework in C++ that integrates advanced simulation techniques and aligns with border research initiatives within the power systems domain. The framework is designed to offer a wide range of functionalities, including component selection, detailed system modeling, and the setup of complex simulation scenarios, which will allow for precise and efficient stability assessments, which are critical for verifying the operational reliability of power systems incorporating both AC and DC networks. This framework will be available as an open-source package, contributing to the wider research and engineering community, and ensuring accessibility for more collaboration and further research in advanced power system analysis.

3.1.1 Objectives

One of the notable features of the proposed framework is performing fast and reliable stability assessments, allowing users to identify and mitigate potential system instabilities quickly. This capability is important for the operational verification of hybrid power systems like MT-HVDC-based systems, which are increasingly important in modern energy infrastructure. Besides, the framework offers detailed representations of harmonic components to increase understanding of the dynamic behavior of power systems. Minor changes in system parameters can significantly impact system stability, making this level of modeling detail crucial for accurate simulation and analysis.
Traditional Electromagnetic Transient tools (EMT), such as PSCAD, offer frequency-scanning methodologies for passive network components to analyze the steady-state harmonic amplification through the network. However, these tools are often limited in their ability to address small-signal stability issues, particularly concerning the dynamic interactions of converter controls.
The proposed framework bridges this gap by incorporating advanced simulation techniques that deliver small-signal representations of converters and their control systems. The goal is to achieve a more accurate assessment of system stability and enable a comprehensive analysis of the dynamic behavior of power systems, including harmonic interactions and stability across a wide range of operating conditions.

3.1.2 Implementation structure

The framework integrates detailed power system models of passive and active components to capture frequency-dependent behaviors across a wide frequency range. It captured a broad range of system dynamics by including detailed mathematical models of components such as transformers, impedance, and admittance, reflecting their frequency-dependent behaviors. For instance, the framework models system impedance with frequency-sensitive properties while accounting for the dynamic behavior of transformers and their effect on power flow.
The proposed framework is designed to effectively model the key components of hybrid AC and DC power systems. The architecture employs a hierarchical class structure, where the Network class manages the organization of buses and elements. The Bus and Element classes exist independently, with Element serving as the base class for a range of power system components, each implemented to reflect its unique role in the system.
The Network class represents the electrical system, managing buses and elements while storing them in hash maps for efficient access. It handles the system's topology by connecting elements to buses and organizing their relationships. The Bus class defines individual buses, which can connect to multiple elements, and the Element class represents generic electrical components capable of computing and storing their admittance matrix (Y_matrix) for system-level impedance and admittance calculations.
The framework employs a flexible structure for multi-phase systems, making it suitable for hybrid AC/DC grids. Additionally, it is designed to compute equivalent impedance between different parts of the system, enhancing its analytical capabilities. Key components derived from the Element class include transmission

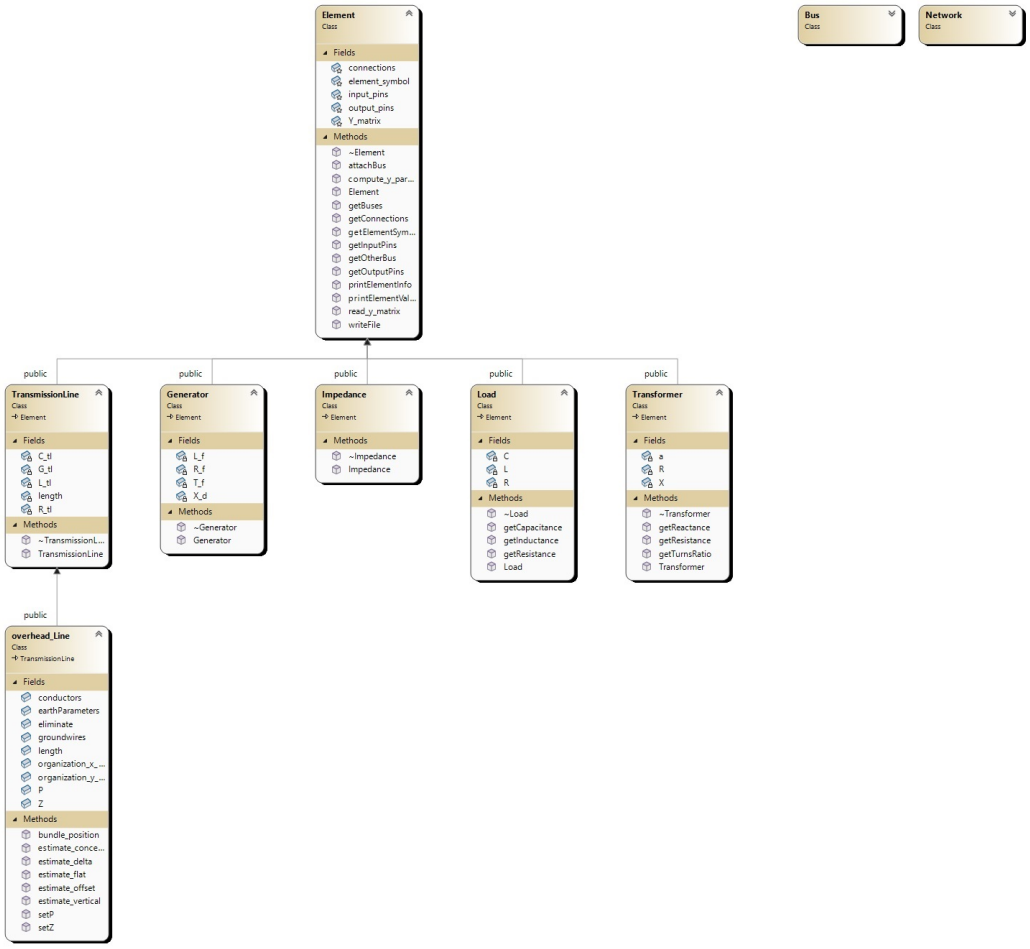

Figure 3.1: Class Diagram Overview

lines, transformers, and active elements such as power electronic devices, each modeled to capture their behavior over a wide frequency range.

The key components, derived from the Element class, include:

- Impedances: This class models the resistive and reactive properties of various system components, such as transformers or passive filters. The impedance class allows for a precise representation of both series and parallel impedance configurations. Based on the number of pins (phases), the admittance matrix (Y_matrix) is computed for each element, either applying a uniform impedance across all phases or assigning individual impedances to each phase. This flexible approach ensures accurate modeling of frequency-dependent behavior across different components in the network. The impedance values can be defined numerically or symbolically, enhancing the model's capability to handle complex system dynamics.

- Admittance: Admittance components are included to represent the shunt elements in the system, capturing both the conductance and susceptance characteristics for accurate network modeling. These components are essential for capturing the behavior of elements such as transformers, lines, and capacitors in a network. The class supports both single-phase and multi-phase systems by allowing users to define admittance values numerically or symbolically. Depending on the input, the admittance matrix is computed as either a single value applied to all phases, a vector for phase-specific admittance, or a full matrix for multi-phase systems. This flexibility ensures an accurate and comprehensive representation of the network's static and dynamic characteristics.

- Loads: The Load class models the demand-side characteristics of the system, representing both resistive (R), inductive (L), and capacitive (C) components in series. This flexible model captures AC and DC consumption points across multiple phases. It allows the definition of uniform or phase-specific R, L, and C values, which ensures accurate modeling of both symmetrical and asymmetrical loads in power systems. The class computes the impedance and admittance matrices for the load, enabling the calculation of complex power flows, accounting for the frequency-dependent behavior of inductive and capacitive elements.
- Transformers: Also derived from the Element class, transformers facilitate voltage level transformations across different parts of the system, essential for maintaining proper power flow and system stability.
- Transmission Lines: Represented by the TransmissionLine class, which further subdivides into Cable and OverheadLine classes. This distinction allows for the specific modeling of different transmission media, each with its electrical characteristics.
- Generators: Modeled as a child of the Element class, generators represent the power-producing units within both AC and DC grids.

This structure enables detailed and scalable modeling of complex hybrid AC/DC power systems, ensuring that every component's interaction is accurately captured and simulated.

Each component has a definition for the number of input and output nodes (or connections).

3.2 Implementation and Computational Modeling of Power System Components

In power system analysis, Y-parameters (admittance parameters) are widely used to represent the relationships between currents and voltages at the terminals of multi-port network components. Y-parameters provide a convenient method for analyzing the steady-state and dynamic behavior of various power system components, including generators, loads, transmission lines, and transformers.

For each component, the Y-parameters describe how the input currents at different ports are related to the input and output voltages, making it easier to assess the impact of each component on the system's overall stability and power flow. This representation is particularly useful for integrating components into larger system models, facilitating the simulation and analysis of hybrid AC and DC power systems.

In this section, we will derive the Y-parameter representation for key power system components, starting with transmission lines.

3.2.1 Impedance

Impedance is created as a series impedance model as shown in Fig. 3.2. Impedance values can be given symbolically (dependent on angular frequency) and as a specific value.

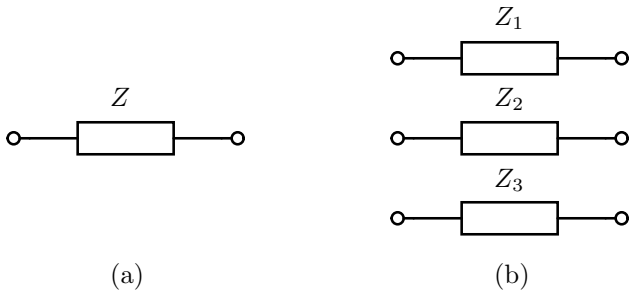

Figure 3.2: Impedance model: (a) single phase or DC; (b) three phase.

For the impedance, Y parameters are calculated as:

$$Y = \begin{bmatrix} \operatorname{diag}\left\{\frac{1}{Z_i}\right\} & -\operatorname{diag}\left\{\frac{1}{Z_i}\right\} \\ -\operatorname{diag}\left\{\frac{1}{Z_i}\right\} & \operatorname{diag}\left\{\frac{1}{Z_i}\right\} \end{bmatrix}, \quad i \in \{1, \ldots, n\}, \tag{3.1}$$

where n is several phases. It must be noted here that the Y matrix is defined as a diagonal one as it is assumed that no mutual couplings exist between the three phases.

Code Implementation To further provide a detailed overview of the implementation of these impedance models in C++, we have formulated the aforesaid Y parameters using one .h and a .cpp file. The derived admittance matrix for the single-phase and the three-phase impedance model provides a clear relationship between the currents and voltages in the system.

This report provides a detailed explanation of the code implementation and structure of the `impedence.h` and the `impedence.cpp` files, which defines the `Impedence` class.

Explanation of 'impedance.h' The 'impedance.h' file defines the class 'Impedance', which is responsible for modelling the impedance and calculating the admittance matrix for a given electrical element. Below is the full content of the 'impedance.h' file:

```
#ifndef _IMPEDANCE_H_
#define _IMPEDANCE_H_

#include "Element.h"

/*
Creates impedance with specified number of input/output pins which represent phases.
The admittance expression has to be given in Omega and can have both numerical and
symbolic value (example: `z = s-2`). Depending on the provided vector of impedance values,
we differ two cases. Namely, we create only series impedance for each phase with values:
-In the case of 1\times1 vector, impedance value is given to all diagonal impedance entries.
-In the case of `pins` elements, they are representing diagonal entries of impedance.
*/

class Impedance : public Element {
public:
    /*
     * Constructor: Impedance
     *
     * Constructs the impedance model with the given symbolic name, number of pins (phases),
     * and a matrix of impedance values. It calculates the admittance matrix based on these values.
     *
     * Parameters:
```

```
     * - symbol: Symbolic identifier for the impedance element (e.g., Z1, Z2)
     * - pins: Number of input/output pins (phases)
     * - values: DenseMatrix representing the impedance values
     */
    Impedance(const std::string& symbol, int pins, DenseMatrix values);

    // Destructor to handle any clean-up tasks
    ~Impedance();

private:
    // No additional private members in this class, as the behavior is inherited from Element
};

#endif // _IMPEDANCE_H_
```

Listing 1: impedance.h File

Header Guards The header guards, defined by '#ifndef _IMPEDANCE_H_', '#define _IMPEDANCE_H_', and '#endif', ensure that this header file is only included once during compilation, preventing any redefinition errors.

Inclusion of 'Element.h' The '#include "Element.h" statement brings in the definition of the base class 'Element'. The 'Impedance' class inherits from 'Element', making it part of a broader system of electrical elements. This allows the 'Impedance' class to leverage common functionality provided by the 'Element' class, such as common properties for electrical components.

Class Documentation The multi-line comment above the 'Impedance' class provides a clear description of its purpose:

- The class models impedance for a system with a specified number of input/output pins, representing phases.
- The impedance values can be either symbolic or numerical, allowing flexibility for various use cases.
- The impedance values are assigned either uniformly (if a 1×1 vector is given) or as diagonal entries of a matrix (if multiple values are provided).

The 'Impedance' Class The 'Impedance' class inherits from the 'Element' class and defines the impedance behaviour. The constructor takes three parameters:

- **symbol**: A string representing the symbolic name of the impedance element (e.g., Z1, Z2).
- **pins**: The number of input/output pins (phases), representing the connection points of the impedance element.
- **values**: A 'DenseMatrix' containing the impedance values for the different phases. This matrix is used to compute the admittance matrix.

The constructor calculates the admittance matrix based on the impedance values, which is crucial for the analysis of electrical circuits.

Destructor The destructor ' Impedance()' is included to handle any clean-up that might be necessary when an 'Impedance' object is destroyed. Although no specific resources are allocated in this class, the destructor is good practice in case the implementation expands in the future.

Private Members There are no additional private members defined in this class. The necessary properties and methods are inherited from the 'Element' class.

Introduction This report further provides a detailed explanation of the 'impedance.cpp' file, which contains the implementation of the 'Impedance' class defined in 'impedance.h'. The implementation includes the constructor that calculates the admittance matrix ('Y_matrix') for a given impedance and the destructor that handles clean-up when the object is destroyed. The 'Impedance' class inherits from the 'Element' class and models multi-phase impedance elements.

Explanation of 'impedance.cpp' The 'impedance.cpp' file implements the functionality for initializing an impedance object and calculating the corresponding admittance matrix. The code also handles different cases based on the number of impedance values provided and ensures that the correct admittance matrix is constructed for single-phase or multi-phase systems.

```
#include "Impedance.h"

/*
 * Impedance Constructor
 *
 * Initializes the impedance element by assigning the admittance matrix (Y_matrix) based on
 * the number of pins and impedance values provided. Depending on whether one value or multiple
 * values are provided, the constructor creates either identical or phase-specific impedance entries.
 */
Impedance::Impedance(const std::string& symbol, int pins, DenseMatrix values):
    Element(symbol, pins, pins) {
    // Check if impedance values are provided
    if (values.ncols() != 0) // if there are entries
    {
        if (pins > 0) { // Check for valid number of pins
            // Case 1: Single impedance value provided for all phases
            if (values.ncols() == 1) {
                for (int i = 0; i < pins; i++)
                    Y_matrix.set(i, i, div(integer(1), values.get(0, 0)));
            }
            // Case 2: Multiple values provided for each phase's impedance
            else if (values.ncols() == pins) {
                for (int i = 0; i < pins; i++)
                    Y_matrix.set(i, i, div(integer(1), values.get(0, i)));
            }
            // Error: The number of impedance values provided doesn't match the number of phases
            else
                throw invalid_argument("Invalid number of admittance vector entries: " + values.ncols());
        }
    }
    else
        throw invalid_argument("Invalid number of pins, must be greater than 0!");

    // Fill in the complete Y parameters
    for (int i = 0; i < pins; i++)
        for (int j = 0; j < pins; j++) {
            Y_matrix.set(pins + i, j, sub(zero, Y_matrix.get(i, j)));
            Y_matrix.set(pins + i, pins + j, Y_matrix.get(i, j));
            Y_matrix.set(i, pins + j, sub(zero, Y_matrix.get(i, j)));
        }
}


// Destructor
Impedance::~Impedance() {
    // Clean-up if needed (Smart pointers handle most memory management)
}
```

Listing 2: impedance.cpp File

Constructor: 'Impedance' The 'Impedance' constructor is responsible for initializing the impedance element and calculating the corresponding admittance matrix ('Y_matrix'). The constructor accepts the following parameters:

- **symbol**: A string representing the symbolic name of the impedance element (e.g., Z1, Z2).
- **pins**: An integer representing the number of phases (or input/output pins) of the impedance element.
- **values**: A 'DenseMatrix' object representing the impedance values for each phase. Depending on the number of impedance values provided, the constructor handles two cases.

The constructor first checks if any impedance values are provided by checking the number of columns in the 'values' matrix. Then, it checks if the number of pins is greater than 0, ensuring that the impedance element has a valid number of phases.

Case 1: Single Impedance Value (single phase cases) If only one impedance value is provided, the constructor assumes that this value applies to all phases equally. The admittance matrix ('Y_matrix') is calculated as the reciprocal of the impedance value for each diagonal entry, as seen in the following loop:

```
for (int i = 0; i < pins; i++)
    Y_matrix.set(i, i, div(integer(1), values.get(0, 0)));
```

Case 2: Multiple Impedance Values (multi-phase or three-phase cases) If multiple impedance values are provided, each phase gets its own specific impedance value. The constructor iterates through each phase and assigns the reciprocal of the respective impedance value to the diagonal entries of the admittance matrix:

```
for (int i = 0; i < pins; i++)
    Y_matrix.set(i, i, div(integer(1), values.get(0, i)));
```

Error Handling If the number of impedance values provided doesn't match the number of phases, an exception is thrown to handle the error:

```
throw invalid_argument("Invalid number of admittance vector entries: " + values.ncols());
```

Filling the Complete Admittance Matrix After setting the diagonal entries, the constructor proceeds to fill the complete admittance matrix ('Y_matrix'). It ensures symmetry between the input and output phases by setting the corresponding off-diagonal elements. The negative values ensure proper alignment of phase admittances:

```
for (int i = 0; i < pins; i++)
    for (int j = 0; j < pins; j++) {
        Y_matrix.set(pins + i, j, sub(zero, Y_matrix.get(i, j)));
        Y_matrix.set(pins + i, pins + j, Y_matrix.get(i, j));
        Y_matrix.set(i, pins + j, sub(zero, Y_matrix.get(i, j)));
    }
```

Destructor: 'Impedance' The destructor is responsible for clean-up tasks when an 'Impedance' object is destroyed. In this implementation, memory management is primarily handled by smart pointers, so no explicit resource release is required.

3.2.2 Admittance

An admittance is constructed as an element by calling the function:
`admittance(;z :: Union{Int, Float64, Basic, Array{Basic}} = 0, pins :: Int = 0)`.

The function creates an impedance with the specified number of input/output pins. The number of input pins is equal to the number of output pins. The impedance expression `exp` has to be given in Ohms and can have both numerical and symbolic values (example: `z = s-2`). Pins are named: 1.1, 1.2, ..., 1.n and 2.1, 2.2, ..., 2.n, where n is a number of input/output pins.

In the case of a 1×1 impedance, the parameter `z` has only one value.
Example: `impedance(z = 1000, pins = 1)`.

If the impedance is multiport (i.e. if it has a number of input pins and output pins greater than 1), then its value is given as an array `[val]` with one, n or $n \times n$ number of values. When the arrway as length 1, then the impedance is defined with only diagonal nonzero values equal to val. When the array has a length equal to the number of pins, the impedance has only diagonal nonzero values equal to the values in the array. When the array is of size $n^2 \times 1$, the impedance matrix is of size $n \times n$ and all its values are defined accordingly to the array.

In this example, `s` is the Laplace operator. The impedance can be added in the form of a transfer function, which can be either a polynomial or a nonlinear function of `s` and even a noninteger powers of `s`, e.g. a pure time delay may be represented by an exponential function of `s`.

The code below can be used to define the network with its impedances, as depicted in Fig. 3.3, and to generate its bode plot.

Azadeh: Please check if it is a proper image?

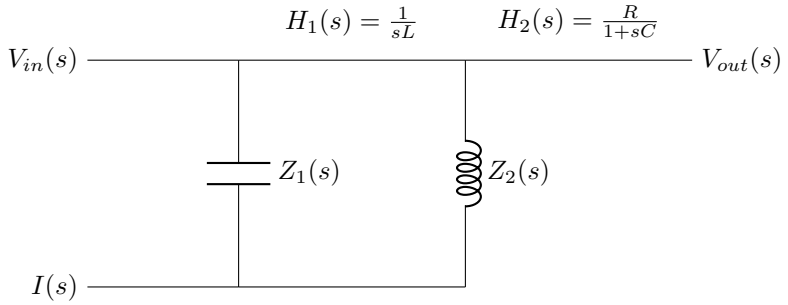

Figure 3.3: Impedance network example with transfer functions involving s.

Introduction This document provides an explanation of the `admittance.h` header file. The `Admittance` class represents an electrical component's admittance in a circuit. It inherits from the base class `Element` and constructs the admittance matrix for a system based on input parameters such as symbolic names, the number of pins (phases), and admittance values. Admittance values can be either numerical or symbolic.

Code Explanation

Header Guards The file starts with header guards, ensuring that the header file is only included once during the compilation process. This prevents redefinition errors.

```
1 #ifndef _ADMITTANCE_H_
2 #define _ADMITTANCE_H_
3
4 #include "Element.h"
```

Listing 3: Header Guards

Admittance Class Overview The `Admittance` class is defined as a derived class from the `Element` class. It handles the creation of the admittance matrix for a given number of input/output pins (phases) based on symbolic or numerical values.

```
/*
Creates admittance with specified number of input/output pins which represent phases.
The admittance expression has to be given in Omega and can have both numerical and
symbolic value (example: `z = s-2`). Depending on the provided vector of admittance values,
we differ three cases. If the number of element pins is greater than 1, admittance can be
represented with vector with one, `pins` or `pins \times pins` number of elements. Namely:
-In the case of 1\times1 vector, admittance has only one value. Then this value is given to all
diagonal admittance entries.
-In the case of `pins` elements, they are representing diagonal entries of admittance.
-In case of `pins \times pins` elements, they are representing all matrix of admittance.
*/

class Admittance : public Element {
```

Listing 4: Class Definition and Description

Constructor The constructor for the `Admittance` class initializes an admittance element based on the provided symbolic name, number of pins (phases), and the admittance values. The values could represent different configurations based on the number of pins (single-phase or multi-phase).

```
    /*
     * Constructor: Admittance
     *
     * Constructs the admittance model with the specified symbolic name, number of pins (phases),
     * and a matrix of admittance values. The values can represent single-phase or multi-phase
     * admittance depending on the input size.
     *
     * Parameters:
     * - symbol: Symbolic identifier for the admittance element (e.g., Y1, Y2)
     * - pins: Number of input/output pins (phases)
     * - values: DenseMatrix representing the admittance values (numerical or symbolic)
     */

    // Unified constructor for single-phase and three-phase systems
    Admittance(const std::string& symbol, int pins, DenseMatrix values);
```

Listing 5: Constructor for Admittance

Parameters

- `symbol`: A symbolic identifier for the admittance element. This is used to distinguish between different admittance components (e.g., `Y1`, `Y2`).
- `pins`: The number of pins (or phases) that the admittance element connects to. This number defines the size of the matrix representing the admittance.
- `values`: A `DenseMatrix` object that holds the actual admittance values. This matrix could represent a single-phase or multi-phase configuration depending on its size.

Destructor The destructor ensures that any memory allocated or resources used by the `Admittance` object are properly released when the object is destroyed. The destructor is declared but not defined within the header file, as the behaviour is inherited from the `Element` class, which likely handles memory management.

```
    // Destructor to handle clean-up tasks
    ~Admittance() override;
```

Listing 6: Destructor for Admittance

Private Members No additional private members are defined in the `Admittance` class. The class relies on the functionality inherited from the `Element` base class.

```
private:
    // No additional private members; behavior is inherited from Element
};
```

Listing 7: Private Members Section

End of Header Guard The file ends with the closing statement for the header guard, ensuring the file is not included multiple times during the compilation process.

```
#endif // _ADMITTANCE_H_
```

Listing 8: End of Header Guard

The `Admittance` class in `admittance.h` provides an interface to model admittance elements in electrical circuits. The class constructor allows for flexible representation of single-phase or multi-phase admittance based on the input values. The class inherits from the `Element` class, which handles core functionalities related to electrical components.

Introduction This document provides an explanation of the `admittance.cpp` file. The `Admittance` class represents an electrical component's admittance and calculates the admittance matrix (`Y_matrix`) based on input parameters such as symbolic names, the number of pins (phases), and admittance values. The class supports different configurations of admittance values for single-phase and multi-phase systems.

Code Explanation

File Inclusion The implementation file begins by including the header file `Admittance.h`, which defines the class interface.

```
#include "Admittance.h"
```

Listing 9: Including the Admittance Header

Constructor The `Admittance` constructor initializes the object by populating the `Y_matrix` based on the provided number of pins and admittance values. It handles three different configurations: a single admittance value, a vector of admittance values for each phase, or a full admittance matrix for multi-phase systems.

```
/*
 * Admittance Constructor
 *
 * Initializes the admittance element by filling the Y_matrix based on the number of pins and
 * the provided admittance values. The constructor supports three configurations:
 * - Case 1: Single admittance value for all phases
 * - Case 2: Vector of admittance values for phase-specific entries
 * - Case 3: Full admittance matrix for multi-phase systems
 */
Admittance::Admittance(const std::string& symbol, int pins, DenseMatrix values) :
Element(symbol, pins, pins) {
```

Listing 10: Admittance Constructor

Input Validation: First, the constructor checks if any admittance values have been provided. If there are no values or the number of pins is not greater than 0, an exception is thrown.

```
    // Check if admittance values are provided
    if (values.ncols() != 0) // if there are entries
    {
        if (pins > 0) { // Check for valid number of pins
```

Listing 11: Input Validation

Case 1: Single Admittance Value (single-phase system): If a single admittance value is provided, it is applied to all diagonal entries in the `Y_matrix`.

```
            // Case 1: Single admittance value, applies to all phases
            if (values.ncols() == 1) {
                for (int i = 0; i < pins; i++)
                    Y_matrix.set(i, i, values.get(0, 0));
            }
```

Listing 12: Case 1: Single Admittance Value

Case 2: Vector of Admittance Values (multi-phase or three-phase cases): If a vector of admittance values is provided, it populates the diagonal of the `Y_matrix`, with each entry corresponding to a specific phase.

```
            // Case 2: Vector of admittance values for each phase (diagonal matrix)
            else if (values.ncols() == pins) {
                for (int i = 0; i < pins; i++)
                    Y_matrix.set(i, i, values.get(0, i));
            }
```

Listing 13: Case 2: Vector of Admittance Values

Case 3: Full Admittance Matrix: If a full admittance matrix is provided, each entry is used to populate the corresponding position in the `Y_matrix`. This is suitable for complex multi-phase systems.

```
            // Case 3: Full matrix of admittance values
            else if (values.ncols() == pins * pins) {
                for (int i = 0; i < pins; i++)
                    for (int j = 0; j < pins; j++)
                        Y_matrix.set(i, j, values.get(0, i + j));
            }
```

Listing 14: Case 3: Full Admittance Matrix

Error Handling: If the provided admittance values do not match any expected configuration (single value, vector, or full matrix), an exception is thrown, indicating an invalid number of values.

```
            // Error: The number of values doesn't match any expected configuration
            else
                throw invalid_argument("Invalid number of admittance vector entries: " + values.ncols());
        }
    }
    else
        throw invalid_argument("Invalid number of pins, must be greater than 0!");
```

Listing 15: Error Handling for Invalid Values

Filling the Complete Y Matrix After processing the admittance values, the constructor fills the complete `Y_matrix` by adjusting the off-diagonal values. This is done to ensure the matrix properly represents the relationships between different phases.

```
    // Fill in the complete Y parameters
    for (int i = 0; i < pins; i++)
        for (int j = 0; j < pins; j++) {
            Y_matrix.set(pins+ i, j, sub(zero, Y_matrix.get(i, j)));
            Y_matrix.set(pins + i, pins+ j, Y_matrix.get(i, j));
            Y_matrix.set(i, pins + j, sub(zero, Y_matrix.get(i, j)));
        }
```

Listing 16: Filling the Y Matrix

Destructor The destructor ensures that any necessary cleanup tasks are handled when the `Admittance` object is destroyed. In this case, manual memory management is not required since the `DenseMatrix` and other standard library components automatically manage memory.

```
// Destructor
Admittance::~Admittance() {
    // No need for manual memory management for DenseMatrix or other standard library components
}
```

Listing 17: Destructor for Admittance

The `Admittance` class's constructor in `admittance.cpp` efficiently initializes the admittance matrix (`Y_matrix`) based on different configurations of admittance values. The constructor is flexible, handling cases with a single value, a vector of values, or a full matrix. The class ensures that errors are properly handled when inputs are invalid, and the destructor takes care of any cleanup tasks when the object is destroyed.

3.2.3 Load

Introduction This report provides a detailed description of a load model undertaken in this project consisting of resistive (R), inductive (L), and capacitive (C) components in series for both single-phase and three-phase systems. The objective is to model the admittance matrix Y for a three-phase system, which characterizes the relationship between currents and voltages in the system.

The model computes the admittance (Y) matrix, which relates the currents I and voltages V in the system (load) through the following relationship:

$$I = YV$$

Series RLC Circuit for Single-Phase We will first start with the single-phase model, derive the Y parameters, and extend to three-phase models. In a single-phase series RLC circuit, the total impedance Z is given by the sum of the resistive, inductive, and capacitive impedances:

$$Z = R + j\omega L - \frac{j}{\omega C}$$

The admittance Y is the inverse of the impedance:

$$Y = \frac{1}{Z} = \frac{1}{R + j\omega L - \frac{j}{\omega C}}$$

The corresponding circuit for a single-phase series RLC load is shown in Figure 3.4.

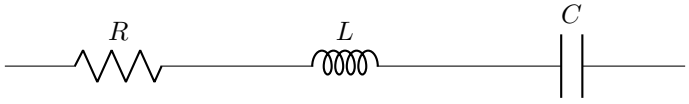

Figure 3.4: Single-Phase Series RLC Load

Three-Phase Series RLC Circuit Model For a balanced three-phase system, we assume that each phase has its own resistive, inductive, and capacitive components, and these components are connected in series. For the three-phase case, it is assumed that there exists no mutual couplings between the loads of the three phases.

For phase a, phase b, and phase c, the impedance for each phase is defined as:

$$Z_a = R_a + j\omega L_a - \frac{j}{\omega C_a}$$

$$Z_b = R_b + j\omega L_b - \frac{j}{\omega C_b}$$

$$Z_c = R_c + j\omega L_c - \frac{j}{\omega C_c}$$

Admittance Matrix for Three-Phase System The admittance matrix Y for a three-phase system is a 3×3 matrix that relates the current vector $I = [I_a, I_b, I_c]^T$ and the voltage vector $V = [V_a, V_b, V_c]^T$ as follows:

$$\begin{bmatrix} I_a \\ I_b \\ I_c \end{bmatrix} = \begin{bmatrix} Y_{aa} & Y_{ab} & Y_{ac} \\ Y_{ba} & Y_{bb} & Y_{bc} \\ Y_{ca} & Y_{cb} & Y_{cc} \end{bmatrix} \begin{bmatrix} V_a \\ V_b \\ V_c \end{bmatrix}$$

For an uncoupled three-phase system, the off-diagonal elements Y_{ab}, Y_{ac}, Y_{ba}, etc., are zero, and the diagonal elements are the admittance of the respective phases.

Balanced Three-Phase System For a balanced three-phase system, we assume that the impedance of each phase is the same:

$$R_a = R_b = R_c = R, \quad L_a = L_b = L_c = L, \quad C_a = C_b = C_c = C$$

The diagonal elements of the admittance matrix are:

$$Y_{aa} = Y_{bb} = Y_{cc} = \frac{1}{R + j\omega L - \frac{j}{\omega C}}$$

This simplifies the admittance matrix for a balanced three-phase system to:

$$Y = \begin{bmatrix} Y_{aa} & 0 & 0 \\ 0 & Y_{aa} & 0 \\ 0 & 0 & Y_{aa} \end{bmatrix} = \begin{bmatrix} \frac{1}{R+j\omega L-\frac{j}{\omega C}} & 0 & 0 \\ 0 & \frac{1}{R+j\omega L-\frac{j}{\omega C}} & 0 \\ 0 & 0 & \frac{1}{R+j\omega L-\frac{j}{\omega C}} \end{bmatrix}$$

It must be noted here that for most of the cases, this Y matrix for the three-phase load is diagonal as the mutual coupling between phases is negligible.

Code Implementation To further provide a detailed overview of the implementation of these load models in C++, we have formulated the aforesaid Y parameters using one .h and a .cpp file. The derived admittance matrix provides a clear relationship between the currents and voltages in the system. The implementation in C++ allows flexibility in handling both single-phase and three-phase loads, making it suitable for various power system applications.

This report provides a detailed explanation of the code implementation and structure of the `load.h` and the `load.cpp` files, which defines the `Load` class. The class models a three-phase series RLC (Resistor-Inductor-Capacitor) load, allowing the user to specify resistance, inductance, and capacitance values either for each phase separately or for a balanced load scenario.

The `Load` class represents an electrical load with resistive, inductive, and capacitive components connected in series. It is designed to handle multiple phases, where each phase can have distinct values for resistance (`R`), inductance (`L`), and capacitance (`C`). The `Load` object is derived from the `Element` base class and offers methods to retrieve the RLC values for each phase.

Class Definition in load.h The header file defines the interface of the `Load` class. It includes a constructor for initialization, destructor, and getter methods for resistance, inductance, and capacitance for each phase.

Constructor Declaration The constructor takes in three parameters: a symbol (`std::string`), an integer representing the number of pins (or phases), and a `std::vector<double>` containing the R, L, and C values.

```
/*
Creates load with resistive, inductive and capacitive components in series. Its constructor
gets information about pin number and furthermore, R, L, C values. These values can be given
as single value per each component R, L, C, and thus, input 3 values as vector. Or there can be
a separate value per each component R, L, C per pin/phase and thus, it gets 3 * pins input
values.
*/
Load::Load(const std::string& symbol, int pins, std::vector<double> values);
```

Getter Methods The getter methods allow users to retrieve the resistance, inductance, and capacitance values for a specific phase. These methods check if the requested phase index is valid. If the index is invalid, an exception is thrown.

```
// Getter for resistance per phase
double getResistance(int phase) const;

// Getter for inductance per phase
```

```
double getInductance(int phase) const;

// Getter for capacitance per phase
double getCapacitance(int phase) const;
```

Private Data Members The class maintains three private `std::vector<double>` members to store the values of resistance (`R`), inductance (`L`), and capacitance (`C`) for each phase.

```
private:
    std::vector<double> R; // Resistance for each phase
    std::vector<double> L; // Inductance for each phase
    std::vector<double> C; // Capacitance for each phase
```

Implementation in load.cpp The 'Load. cpp' file is part of a larger project that models electrical components using a class-based structure. This file specifically implements a load element capable of handling resistive, inductive, and capacitive components in series.

Load Class Overview The 'Load' class inherits from the 'Element' class, and its primary function is to define the parameters of an electrical load, including resistance (R), inductance (L), and capacitance (C) for multiple phases.

Constructor The constructor of the 'Load' class initializes the load parameters based on the input values. The following code snippet outlines the constructor's implementation:

```
Load::Load(const std::string& symbol, int pins, std::vector<double> values)
    : Element(symbol, pins, pins) {
    if (pins == 0)
        throw std::invalid_argument("Invalid number of pins, must be greater than 0!");

    int capacity = values.capacity();
    if (capacity == 3) {
        R = std::vector<double>(pins, values[0]);
        L = std::vector<double>(pins, values[1]);
        C = std::vector<double>(pins, values[2]);
    }
    else if (capacity == 3 * pins) {
        for (int i = 0; i < pins; i++) {
            R.push_back(values[i]);
            L.push_back(values[pins + i]);
            C.push_back(values[2 * pins + i]);
        }
    }
    else
        throw std::invalid_argument("Invalid number of values, it should be equal to the number of pins, or 3 x
            number of pins.");
}
```

Listing 18: Load Class Constructor

Parameter Initialization The constructor checks the validity of the input parameters and initializes the R, L, and C values based on the number of pins specified. The constructor throws exceptions if the number of pins is invalid or if the size of the values vector does not match the expected capacity.

Parameter Validation After initializing the parameters, the constructor validates them to ensure they are non-negative and properly initialized:

```
for (int i = 0; i < pins; ++i) {
    if (R[i] == 0 && L[i] == 0 && C[i] == 0) {
        std::cerr << "Load parameters not initialized correctly for phase " << i + 1 << "!" << std::endl;
```

```
        return;
    }
    else if (R[i] < 0 || L[i] < 0 || C[i] < 0) {
        std::cerr << "Load parameters not initialized correctly for phase " << i + 1 << "!" << std::endl;
        return;
    }
    else {
        std::cerr << "Load parameters initialized correctly for phase " << i + 1 << "!" << std::endl;
    }
}
```

Listing 19: Load Parameter Validation

This part of the code ensures that any load with invalid parameters will raise an error message indicating which phase is problematic.

Y-Parameter Calculation The next part of the constructor calculates the Y parameters for each phase based on the initialized R, L, and C values:

```
for (int i = 0; i < input_pins; ++i) {
    RCP<const Basic> R_val = real_double(R[i]);
    RCP<const Basic> L_val = real_double(L[i]);
    RCP<const Basic> C_val = real_double(C[i]);

    // Calculate the impedance part from inductance: $j \omega L$
    RCP<const Basic> j_omega_L = mul(j, mul(omega, L_val));
    // Calculate the capacitive impedance: $\frac{j}{\omega C}$
    RCP<const Basic> j_omega_C = mul(j, div(real_double(-1), mul(omega, C_val)));

    // Impedance
    RCP<const Basic> Z_RLC = add(add(R_val, j_omega_L), j_omega_C);

    // Admittance
    Y_matrix.set(i,i, div(real_double(1), Z_RLC));
}
```

Listing 20: Y-Parameter Calculation

This segment computes the total impedance of the load and then derives the corresponding admittance for each phase, which is stored in the 'Y_matrix'.

Complete Y_matrix Population Finally, the code populates the complete Y-parameter matrix based on the previously calculated values for any number of phases as shown. It must be noted as there is no mutual couplings between the loads, hence the derived Y_matrix remains a diagonal one.

```
for (int i = 0; i < pins; i++)
    for (int j = 0; j < pins; j++) {
        Y_matrix.set(pins + i, j, sub(zero, Y_matrix.get(i, j)));
        Y_matrix.set(pins + i, pins + j, Y_matrix.get(i, j));
        Y_matrix.set(i, pins + j, sub(zero, Y_matrix.get(i, j)));
    }
}
```

Listing 21: Complete Y-Parameter Matrix Population

This loops through the matrix to ensure all the appropriate Y parameters are set, including the off-diagonal elements, reflecting the relationships between different phases.

3.2.4 Transformer - Single Phase

Introduction A single-phase transformer is an essentially electrical device that transfers electrical energy between two or more circuits through electromagnetic induction. These transformers are commonly employed in power distribution networks, residential and commercial buildings, and various industrial applications, such as supplying power to lighting systems, heating equipment, and small motors. The possible ratings of single-phase transformers typically range from a few VA (volt-amperes) for small transformers used in electronic applications to several kVA (kilovolt-amperes) for larger transformers in distribution systems.

Mathematical Modeling of a Single-Phase Transformer

Transformer Basics A single-phase transformer consists of a primary winding and a secondary winding. The voltage and current relationships can be expressed as:

$$\frac{V_p}{V_s} = \frac{N_p}{N_s} = a$$

$$\frac{I_s}{I_p} = \frac{N_p}{N_s} = \frac{1}{a}$$

Where: - V_p = Primary Voltage - V_s = Secondary voltage - I_p = Primary current - I_s = Secondary current - N_p = Number of turns in the primary winding - N_s = Number of turns in the secondary winding - a = Turns ratio

Y Parameters For a two-port network, the Y parameters can be defined as:

$$I = YV$$

Where I and V are vectors of currents and voltages at the ports. The Y parameters are derived from the impedance matrix and can be expressed in terms of the primary and secondary parameters.

Equations for Y Parameters The Y parameters for a transformer can be expressed as:

$$Y_{11} = \frac{1}{Z_p}, \quad Y_{12} = -\frac{Y_{11}}{a^2}$$

$$Y_{21} = -Y_{12}, \quad Y_{22} = \frac{1}{Z_s}$$

Where Z_p and Z_s are the equivalent impedances seen from the primary and secondary sides.

Calculating the Y Parameters for the Single-Phase Transformer To derive the Y parameters for the single-phase transformer, we follow these steps:

Determine Equivalent Impedances 1. **Primary Impedance Z_p**: The primary impedance can be calculated using the resistances and reactances of the primary winding:

$$Z_p = R_p + jX_p$$

where R_p is the resistance and X_p is the reactance of the primary winding.

2. **Secondary Impedance Z_s**: Similarly, the secondary impedance is determined by:

$$Z_s = R_s + jX_s$$

where R_s is the resistance and X_s is the reactance of the secondary winding.

Calculate Y Parameters With the equivalent impedances determined, the Y parameters can be calculated using the previously defined relationships:

1. **Calculate Y_{11}**:
$$Y_{11} = \frac{1}{Z_p}$$
2. **Calculate Y_{12}**:
$$Y_{12} = -\frac{Y_{11}}{a^2}$$
3. **Calculate Y_{21}**:
$$Y_{21} = -Y_{12}$$
4. **Calculate Y_{22}**:
$$Y_{22} = \frac{1}{Z_s}$$

This methodology allows for the computation of the Y parameters based on the physical characteristics of the transformer and its winding impedances.

Implementation in C++ This report provides a detailed explanation of the `Transformer` class implemented in C++. The `Transformer` class is a subclass of `Element` and is used to model the behavior of transformers in power systems.

Code Overview Below is the code for the `transformer.h` header file:

```cpp
#ifndef TRANSFORMER_H
#define TRANSFORMER_H

#include "Element.h"

class Transformer : public Element {
public:
    // Constructor to initialize the Transformer with a given symbol, number of pins, and values
    Transformer(const std::string& symbol, int pins, const std::vector<double>& values);

    ~Transformer() override;

    double getResistance(int winding) const {
        if (winding >= 0 && winding < R.size()) {
            return R[winding];
        }
        throw std::out_of_range("Invalid winding index");
    }

    double getReactance(int winding) const {
        if (winding >= 0 && winding < X.size()) {
            return X[winding];
        }
        throw std::out_of_range("Invalid winding index");
    }

    double getTurnsRatio() const { return a; }

private:
    std::vector<double> R; // Resistances for primary and secondary windings
    std::vector<double> X; // Reactances for primary and secondary windings
    double a; // Turns ratio
};

#endif // TRANSFORMER_H
```

Listing 22: Transformer Class Header File

Class Definition

Header Guards The header guard ensures that the contents of this header file are included only once during compilation:

```
#ifndef TRANSFORMER_H
#define TRANSFORMER_H
```

Includes The class includes the header file for the `Element` class, from which it inherits:

```
#include "Element.h"
```

Class Declaration The `Transformer` class inherits from the `Element` class:

```
class Transformer : public Element {
```

Public Methods

- `Transformer(const std::string& symbol, int pins, const std::vector<double>& values)`: Constructor that initializes the transformer with a symbol, the number of pins, and values.
- `Transformer() override`: Destructor that overrides the base class destructor.
- `double getResistance(int winding) const`: Returns the resistance of the specified winding. It checks for valid winding indices and throws an exception if invalid.
- `double getReactance(int winding) const`: Returns the reactance of the specified winding. Similar to `getResistance`, it checks for valid indices.
- `double getTurnsRatio() const`: Returns the turns ratio of the transformer.

Private Member Variables

- `std::vector<double> R`: A vector that stores the resistances for both primary and secondary windings.
- `std::vector<double> X`: A vector that stores the reactances for both primary and secondary windings.
- `double a`: A variable that represents the turns ratio of the transformer.

Closing Header Guard The header guard is closed to prevent multiple inclusions:

```
#endif // TRANSFORMER_H
```

Implementation File: transformer.cpp The 'transformer.cpp' file implements the methods declared in the header file. Below is a brief explanation of its components:

Below is the code for the `transformer.cpp` file:

```
#include "Transformer.h"

// Constructor
Transformer::Transformer(const std::string& symbol, int pins, const std::vector<double>& values)
    : Element(symbol, pins, pins) {

    if (values.size() == 5) {
        R = { values[0], values[2] }; // Primary and secondary resistances
        X = { values[1], values[3] }; // Primary and secondary reactances
        a = values[4]; // Turns ratio
```

```
    }
    else {
        throw std::invalid_argument("Invalid number of values, must be 5 (R_primary, X_primary, R_secondary,
            X_secondary, a)!");
    }

    // Check if the values are properly initialized
    for (int i = 0; i < 2; ++i) {
        if (R[i] == 0 || X[i] == 0 || a == 0) {
            std::cerr << "Transformer parameters not initialized correctly for winding " << i + 1 << "!" << std
                ::endl;
            return;
        }
    }

    // Y parameters matrix
    // Define symbolic resistances and reactances for primary and secondary windings
    // Primary side
    RCP<const Basic> R_p = real_double(R[0]);
    RCP<const Basic> X_p = real_double(X[0]);

    // Secondary side
    RCP<const Basic> R_s = real_double(R[1]);
    RCP<const Basic> X_s = real_double(X[1]);
    RCP<const Basic> a_val = real_double(a); // Turns ratio symbol

    // Compute the primary and secondary admittances
    RCP<const Basic> Y_p = div(real_double(1), add(R_p, mul(j, X_p)));
    RCP<const Basic> Y_s = div(real_double(1), add(R_s, mul(j, X_s)));

    // Compute Y-parameters
    for (int i = 0; i < pins; i++) {
        Y_matrix.set(i, i, Y_p); // Y11
        Y_matrix.set(i, pins + i, mul(real_double(-a), Y_p)); // Y12
        Y_matrix.set(pins + i, i, Y_matrix.get(0, 1)); // Y21 (symmetrical to Y12)
        Y_matrix.set(pins + i, pins + i, Y_s); // Y22
    }
}

// Destructor
Transformer::~Transformer() {
    std::cout << "Transformer object for " << getElementSymbol() << " destroyed." << std::endl;
}
```

Listing 23: Transformer Class Implementation

Class Implementation

Includes The implementation starts with including the header file for the `Transformer` class:

```
#include "Transformer.h"
```

Constructor The constructor initializes the transformer with a given symbol, number of pins, and values:

```
Transformer::Transformer(const std::string& symbol, int pins, const std::vector<double>& values)
    : Element(symbol, pins, pins) {
```

It also checks if the provided values are correct:

```
if (values.size() == 5) {
    R = { values[0], values[2] }; // Primary and secondary resistances
    X = { values[1], values[3] }; // Primary and secondary reactances
    a = values[4]; // Turns ratio
}
else {
```

```
    throw std::invalid_argument("Invalid number of values, must be 5 (R_primary, X_primary, R_secondary,
        X_secondary, a)!");
}
```

Parameter Validation The constructor validates that none of the parameters are zero and represents that all the parameters are defined and initialised correctly.

```
for (int i = 0; i < 2; ++i) {
    if (R[i] == 0 || X[i] == 0 || a == 0) {
        std::cerr << "Transformer parameters not initialized correctly for winding " << i + 1 << "!" << std::
            endl;
        return;
    }
}
```

Y-Parameters Calculation The constructor defines symbolic resistances and reactances and computes the admittance for the primary and secondary sides:

```
// Define symbolic resistances and reactances for primary and secondary windings
RCP<const Basic> R_p = real_double(R[0]);
RCP<const Basic> X_p = real_double(X[0]);
RCP<const Basic> R_s = real_double(R[1]);
RCP<const Basic> X_s = real_double(X[1]);
RCP<const Basic> a_val = real_double(a); // Turns ratio symbol

// Compute the primary and secondary admittances
RCP<const Basic> Y_p = div(real_double(1), add(R_p, mul(j, X_p)));
RCP<const Basic> Y_s = div(real_double(1), add(R_s, mul(j, X_s)));
```

The Y-parameters are then computed and stored in a matrix Y_matrix. It must be noted here that for this formulation, it is assumed that there is no mutual coupling existing between the phases (as in the code the number of pins can be modelled as the number of phases this model is intended to work for), hence this Y_matrix for the multiphase system can be defined as a diagonal one. This represents a simplistic case for analysis and this model is more suitable for a single phase transformer case.

```
// Compute Y-parameters
for (int i = 0; i < pins; i++) {
    Y_matrix.set(i, i, Y_p); // Y11
    Y_matrix.set(i, pins + i, mul(real_double(-a), Y_p)); // Y12
    Y_matrix.set(pins + i, i, Y_matrix.get(0, 1)); // Y21 (symmetrical to Y12)
    Y_matrix.set(pins + i, pins + i, Y_s); // Y22
}
```

Destructor The destructor outputs a message when a transformer object is destroyed:

```
Transformer::~Transformer() {
    std::cout << "Transformer object for " << getElementSymbol() << " destroyed." << std::endl;
}
```

3.2.5 Transformers - Three Phase Models

Introduction Three-phase Δ-Δ transformers are widely used in industrial and commercial power systems. They provide several advantages, including high power handling capability and robust performance in balancing loads across phases. It must be noted that the mutual impedance and cross-couplings between the phases are neglected in this model.

The Δ-Δ configuration offers the benefit of reduced harmonic distortion and improved voltage regulation. This configuration is commonly found in applications where substantial power is required, such as:

- **Industrial Applications:** Power distribution in manufacturing plants, motor drives, and heavy machinery operations.
- **Commercial Buildings:** Supply to large commercial facilities, office complexes, and shopping malls.
- **Power Generation:** Interfacing with generators and renewable energy systems, such as wind and solar farms.

The possible ratings for Δ-Δ transformers typically range from several kilovolt-amperes (kVA) to several megavolt-amperes (MVA), with common ratings including:

- **Small Ratings:** 50 kVA to 500 kVA for small industrial and commercial applications.
- **Medium Ratings:** 500 kVA to 5 MVA for larger industrial loads.
- **Large Ratings:** Above 5 MVA for utility-scale power distribution and large industrial processes.

Mathematical Modeling The modelling of a three-phase Δ-Δ transformer is essential for understanding its behaviour in electrical circuits. The primary focus is on calculating the Y parameters, which relate to the currents and voltages at the transformer terminals.

Y Parameters Calculation The admittance matrix Y is a representation of the relationship between current and voltage in a multiport system. For a Δ-Δ transformer, we consider the primary and secondary windings. It must be noted here that the subscript p in this report describes the primary side of the transformer while the subscript s denotes the secondary side of the transformer.

Admittance Calculation The admittance Y for each winding can be calculated using the formula:

$$Y = \frac{1}{R + jX}$$

where R is the resistance and X is the reactance of the winding. The admittance for each phase can be represented as:

$$Y_{p_i} = \frac{1}{R_{p_i} + jX_{p_i}} \quad \text{for } i = 1, 2, 3$$

$$Y_{s_i} = \frac{1}{R_{s_i} + jX_{s_i}} \quad \text{for } i = 1, 2, 3$$

where Y_{p_i} and Y_{s_i} are the admittances of the primary and secondary windings, respectively.

Constructing the Y Matrix For a three-phase Δ-Δ transformer, the Y parameters matrix can be represented as a 6x6 matrix, as shown:

$$Y = \begin{bmatrix} Y_{p1} & Y_{p1} & 0 & -aY_{p1} & 0 & 0 \\ Y_{p1} & Y_{p2} & -Y_{p1} & 0 & 0 & 0 \\ 0 & -Y_{p1} & Y_{p3} & 0 & 0 & 0 \\ -aY_{p1} & 0 & 0 & Y_{s1}e^{-j\phi} & Y_{s1}e^{-j\phi} & 0 \\ 0 & 0 & 0 & Y_{s1}e^{-j\phi} & Y_{s2}e^{-j\phi} & -Y_{s1}e^{-j\phi} \\ 0 & 0 & 0 & 0 & -Y_{s1}e^{-j\phi} & Y_{s3}e^{-j\phi} \end{bmatrix}$$

where: Y_{p1}, Y_{p2}, Y_{p3} are the primary winding admittances, Y_{s1}, Y_{s2}, Y_{s3} are the secondary winding admittances, a is the turns ratio, $e^{-j\phi}$ is the phase shift factor, with ϕ representing the phase angle difference between the primary and secondary voltages. In this report, we have assumed that all three phases are balanced and with equal impedances i.e. with equal resistance and reactances. Furthermore, this report extends to the single-phase transformer model where the phase shift factors are not considered explicitly for the three-phase case.

Voltage and Current Relationships

Voltage Relationships In a three-phase delta-delta transformer, the phase voltages on the primary side are equal, and they can be expressed as follows:

$$V_{p1} = V_{p2} = V_{p3} = V_p$$

The secondary side voltages can be defined with a phase shift factor ($e^{-j\phi}$) as:

$$\begin{aligned} V_{s1} &= \frac{V_{p1}}{a} e^{-j\phi} \\ V_{s2} &= \frac{V_{p2}}{a} e^{-j(\phi + \frac{2\pi}{3})} \\ V_{s3} &= \frac{V_{p3}}{a} e^{-j(\phi - \frac{2\pi}{3})} \end{aligned}$$

where, V_p represents the balanced phase voltage. Here, a represents the turns ratio of the transformer.

Current Relationships The primary side currents can be represented as:

$$\begin{aligned} I_{p1} &= I_{L1} \\ I_{p2} &= I_{L2} \\ I_{p3} &= I_{L3} \end{aligned}$$

where I_{L1}, I_{L2}, I_{L3} represent the three phase balanced line currents. The currents on the secondary side, incorporating the phase shifts, can be expressed as:

$$\begin{aligned} I_{s1} &= a \cdot I_{p1} e^{j\phi} \\ I_{s2} &= a \cdot I_{p2} e^{j(\phi - \frac{2\pi}{3})} \\ I_{s3} &= a \cdot I_{p3} e^{j(\phi + \frac{2\pi}{3})} \end{aligned}$$

The modified voltage and current relationships in a three-phase delta-delta transformer with phase shifts can be summarized as follows:

Voltage Relationships

$$\begin{aligned} V_{s1} &= \frac{V_{p1}}{a} e^{-j\phi} \\ V_{s2} &= \frac{V_{p2}}{a} e^{-j(\phi + \frac{2\pi}{3})} \\ V_{s3} &= \frac{V_{p3}}{a} e^{-j(\phi - \frac{2\pi}{3})} \end{aligned}$$

Current Relationships

$$I_{s1} = a \cdot I_{p1} e^{j\phi}$$
$$I_{s2} = a \cdot I_{p2} e^{j(\phi - \frac{2\pi}{3})}$$
$$I_{s3} = a \cdot I_{p3} e^{j(\phi + \frac{2\pi}{3})}$$

Implementation in Code This report provides a comprehensive explanation of the header file `transformer_Delta_Delta.h`, which defines the `Transformer_Delta_Delta` class for a three-phase delta-delta transformer in a C++ project. The class inherits from a base class `Element` and encapsulates the properties and methods relevant to the operation of a delta-delta transformer.

Header File Overview The contents of the `transformer_delta_delta.h` file are shown below:

```cpp
#ifndef TRANSFORMER_DELTA_DELTA_H
#define TRANSFORMER_DELTA_DELTA_H

#include "Element.h"

class TransformerDeltaDelta : public Element {
public:
    // Constructor to initialize the Transformer with a given symbol, number of pins, and values (with phase lag)
    TransformerDeltaDelta(const std::string& symbol, int pins, const std::vector<double>& values);

    ~TransformerDeltaDelta() override;

    double getResistance(int winding) const {
        if (winding >= 0 && winding < R.size()) {
            return R[winding];
        }
        throw std::out_of_range("Invalid winding index");
    }

    double getReactance(int winding) const {
        if (winding >= 0 && winding < X.size()) {
            return X[winding];
        }
        throw std::out_of_range("Invalid winding index");
    }

    double getTurnsRatio() const { return a; }

    double getPhaseLag() const { return phaseLag; } // New method to get phase lag

private:
    std::vector<double> R; // Resistances for primary and secondary windings
    std::vector<double> X; // Reactances for primary and secondary windings
    double a; // Turns ratio
    double phaseLag; // Phase lag to incorporate in the Y parameters
};

#endif // TRANSFORMER_DELTA_DELTA_H
```

Code Explanation

Include Guards The first lines of the file include guards:

```cpp
#ifndef TRANSFORMER_DELTA_DELTA_H
#define TRANSFORMER_DELTA_DELTA_H
```

These preprocessor directives ensure that the header file is included only once during the compilation process. If `TRANSFORMER_DELTA_DELTA_H` is already defined, the contents of the file will not be processed again, preventing redefinition errors.

Inclusion of Dependencies The next line includes the header file for the `Element` class:

```
#include "Element.h"
```

This inclusion allows the `Transformer_Delta_Delta` class to inherit properties and methods from the `Element` class, enabling it to function as a specialized type of element within the larger context of the application.

Class Definition The class is declared as follows:

```
class TransformerDeltaDelta : public Element {
```

This line indicates that `Transformer_Delta_Delta` is derived from the `Element` class, allowing it to utilize the functionalities provided by `Element` and facilitating polymorphism.

Public Methods The public section of the class contains the following methods:

Constructor The constructor of the 'Transformer_Delta_Delta' class initializes the transformer parameters based on the input values. The following code snippet outlines the constructor's implementation:

```
TransformerDeltaDelta(const std::string& symbol, int pins, const std::vector<double>& values);
```

The constructor initializes a transformer object with the specified parameters:

- `symbol`: A string representing the transformer's identifier.
- `pins`: An integer indicating the number of pins (connections) for the transformer.
- `values`: A vector of doubles containing the resistances, reactances, turns ratio, and phase lag for the transformer.

Destructor The destructor cleans up any resources when the transformer object is destroyed. The `override` keyword indicates that this destructor overrides a virtual destructor in the base `Element` class.

```
~TransformerDeltaDelta() override;
```

Methods to Access Parameters The class provides methods to access the transformer's parameters:

```
double getResistance(int winding) const;
```

This method returns the resistance of the specified winding, checking for valid indices.

```
double getReactance(int winding) const;
```

This method similarly returns the reactance for a specified winding, with bounds checking.

```
double getTurnsRatio() const { return a; }
```

This method returns the turns ratio a of the transformer.

```
double getPhaseLag() const { return phaseLag; }
```

This new method returns the phase lag associated with the transformer, essential for accurately capturing the voltage-current relationship in the admittance matrix.

Private Member Variables The private section contains member variables that store the transformer's parameters:

```
std::vector<double> R; // Resistances for primary and secondary windings
std::vector<double> X; // Reactances for primary and secondary windings
double a; // Turns ratio
double phaseLag; // Phase lag to incorporate in the Y parameters
```

- `R`: A vector that holds the resistances of both the primary and secondary windings.
- `X`: A vector that holds the reactances for both windings.
- `a`: The turns ratio of the transformer.
- `phaseLag`: A phase lag value that aids in the calculation of Y parameters in the admittance matrix.

Further, this report provides a comprehensive explanation of the C++ code implemented for modelling a three-phase delta-delta transformer in the file `Transformer_delta_delta.cpp`. The transformer is modelled as an electrical element with specified parameters such as resistances, reactances, turns ratio, and phase lag.

Header File Inclusion Below is the code for the `Transformer_delta_delta.cpp` file:

```
#include "transformer_delta_delta.h"
#include <cmath>

// Constructor
TransformerDeltaDelta::TransformerDeltaDelta(const std::string& symbol, int pins, const std::vector<double>&
     values)
    : Element(symbol, pins, pins) {

    if (values.size() == 6) {
        R = { values[0], values[2] }; // Primary and secondary resistances
        X = { values[1], values[3] }; // Primary and secondary reactances
        a = values[4]; // Turns ratio
        phaseLag = values[5]; // Phase lag
    }
    else {
        throw std::invalid_argument("Invalid number of values, must be 6 (R_primary, X_primary, R_secondary,
             X_secondary, a, phaseLag)!");
    }

    // Y parameters matrix for the three-phase delta-delta transformer
    // We use a 6x6 Y matrix to represent the admittance between the 3 primary and 3 secondary windings.

    RCP<const Basic> R_p = real_double(R[0]);
    RCP<const Basic> X_p = real_double(X[0]);
    RCP<const Basic> R_s = real_double(R[1]);
    RCP<const Basic> X_s = real_double(X[1]);
    RCP<const Basic> a_val = real_double(a); // Turns ratio symbol

    RCP<const Basic> phaseFactor = exp(mul(neg(j), real_double(phaseLag))); // Phase shift

    // Compute primary and secondary admittances
    RCP<const Basic> Y_p = div(real_double(1), add(R_p, mul(j, X_p)));
    RCP<const Basic> Y_s = div(real_double(1), add(R_s, mul(j, X_s)));

    // Populate the Y matrices for primary and secondary sides (delta configuration)
    DenseMatrix Y_delta_p(3, 3);
    DenseMatrix Y_delta_s(3, 3);

    Y_delta_p.set(0, 0, Y_p);
    Y_delta_p.set(0, 1, mul(real_double(-1), Y_p));
    Y_delta_p.set(0, 2, real_double(0));

    Y_delta_p.set(1, 0, mul(real_double(-1), Y_p));
```

```
    Y_delta_p.set(1, 1, Y_p);
    Y_delta_p.set(1, 2, mul(real_double(-1), Y_p));

    Y_delta_p.set(2, 0, real_double(0));
    Y_delta_p.set(2, 1, mul(real_double(-1), Y_p));
    Y_delta_p.set(2, 2, Y_p);

    Y_delta_s.set(0, 0, Y_s);
    Y_delta_s.set(0, 1, mul(real_double(-1), Y_s));
    Y_delta_s.set(0, 2, real_double(0));

    Y_delta_s.set(1, 0, mul(real_double(-1), Y_s));
    Y_delta_s.set(1, 1, Y_s);
    Y_delta_s.set(1, 2, mul(real_double(-1), Y_s));

    Y_delta_s.set(2, 0, real_double(0));
    Y_delta_s.set(2, 1, mul(real_double(-1), Y_s));
    Y_delta_s.set(2, 2, Y_s);

    // Assign submatrices to the Y_matrix (6x6)
    for (int i = 0; i < 3; ++i) {
        for (int j = 0; j < 3; ++j) {
            Y_matrix.set(i, j, Y_delta_p.get(i, j)); // Primary winding
            Y_matrix.set(i + 3, j + 3, Y_delta_s.get(i, j)); // Secondary winding

            // Primary to secondary interaction with phase shift
            Y_matrix.set(i, j + 3, mul(mul(real_double(-a), phaseFactor), Y_delta_p.get(i, j)));

            // Secondary to primary interaction with phase shift
            Y_matrix.set(i + 3, j, mul(mul(real_double(-1), phaseFactor), Y_delta_p.get(i, j)));
        }
    }
}

// Destructor
TransformerDeltaDelta::~TransformerDeltaDelta() {
    std::cout << "TransformerDeltaDelta object for " << getElementSymbol() << " destroyed." << std::endl;
}
```

Listing 24: Transformer three phase Δ-Δ Class Implementation

The code begins by including the necessary header files:

```
#include "transformer_delta_delta.h"
#include <cmath>
```

- `transformer_delta_delta.h`: This file defines the class and its member functions.
- `cmath`: This header is included for mathematical operations.

Constructor Definition The constructor initializes the three phase Δ-Δ transformer parameters and constructs the admittance matrix Y.

Constructor Signature The constructor is defined as follows:

```
TransformerDeltaDelta::TransformerDeltaDelta(const std::string& symbol, int pins, const std::vector<double>&
    values)
    : Element(symbol, pins, pins) {
```

- `symbol`: A string representing the symbol of the transformer.
- `pins`: The number of pins associated with the transformer (should be 6 for a three-phase delta-delta transformer).
- `values`: A vector of doubles containing the transformer parameters.

Parameter Initialization The constructor checks the size of the `values` vector and initializes the transformer parameters:

```
if (values.size() == 6) {
    R = { values[0], values[2] }; // Primary and secondary resistances
    X = { values[1], values[3] }; // Primary and secondary reactances
    a = values[4]; // Turns ratio
    phaseLag = values[5]; // Phase lag
}
else {
    throw std::invalid_argument("Invalid number of values, must be 6 (R_primary, X_primary, R_secondary,
        X_secondary, a, phaseLag)!");
}
```

- R: A vector storing primary and secondary resistances.
- X: A vector storing primary and secondary reactances.
- a: The turns ratio of the transformer.
- `phaseLag`: The phase shift introduced in the transformer model.

An exception is thrown if the number of values is not equal to 6.

Y Parameters Matrix Construction Next, the Y parameters matrix representing the admittance between the primary and secondary windings is constructed. A 6x6 matrix is used for this purpose.

Y Parameters Calculation The admittance for the primary and secondary sides is computed as follows:

```
RCP<const Basic> Y_p = div(real_double(1), add(R_p, mul(j, X_p)));
RCP<const Basic> Y_s = div(real_double(1), add(R_s, mul(j, X_s)));
```

Here, Y_p and Y_s are the admittance values for the primary and secondary sides, respectively.

Delta Configuration Y Matrices The Y matrices for both the primary and secondary sides are constructed in the delta configuration:

```
DenseMatrix Y_delta_p(3, 3);
DenseMatrix Y_delta_s(3, 3);

Y_delta_p.set(0, 0, Y_p);
Y_delta_p.set(0, 1, mul(real_double(-1), Y_p));
Y_delta_p.set(0, 2, real_double(0));

// Repeating for other elements...
```

This process involves filling in a 3x3 matrix for both primary and secondary sides according to the delta configuration rules.

Combining Y Matrices into a 6x6 Matrix After constructing the 3x3 Y matrices for both primary and secondary windings, the code combines them into a single 6x6 Y matrix:

```
for (int i = 0; i < 3; ++i) {
    for (int j = 0; j < 3; ++j) {
        Y_matrix.set(i, j, Y_delta_p.get(i, j)); // Primary winding
        Y_matrix.set(i + 3, j + 3, Y_delta_s.get(i, j)); // Secondary winding

        // Primary to secondary interaction with phase shift
        Y_matrix.set(i, j + 3, mul(mul(real_double(-a), phaseFactor), Y_delta_p.get(i, j)));

        // Secondary to primary interaction with phase shift
        Y_matrix.set(i + 3, j, mul(mul(real_double(-1), phaseFactor), Y_delta_p.get(i, j)));
```

```
    }
}
```

In this segment:

- The upper left 3×3 block represents the primary windings.
- The lower right 3×3 block represents the secondary windings.
- Off-diagonal elements represent the interaction between primary and secondary sides, factoring in the turns ratio and phase shift.

Destructor Definition The destructor is defined to clean up resources when an object of the transformer class is destroyed:

```
TransformerDeltaDelta::~TransformerDeltaDelta() {
    std::cout << "TransformerDeltaDelta object for " << getElementSymbol() << " destroyed." << std::endl;
}
```

This function outputs a message indicating that the transformer object has been destroyed.

Introduction Three-phase Δ-Y transformers are commonly used in power systems for voltage transformation and isolation between different parts of the network. The Y (or star) side typically has its neutral grounded, providing a stable reference voltage. Delta-Y transformers are preferred for many applications due to their ability to handle unbalanced loads and suppress certain harmonics.

This report generalizes the model for a three-phase Δ-Y transformer by including:

- A phase shift factor between the primary (delta) and secondary (Y) windings.
- Mutual and cross-coupling between phases for a realistic representation.

Thus it extends on the three-phase $\Delta - \Delta$ transformer model and incorporates the mutual and the cross-couplings between the three phases.

Applications The Δ-Y configuration is common in:

- **Distribution Networks:** Voltage transformation for supplying loads at lower voltage levels.
- **Power Generation:** Interfacing generators to the grid, especially when grounding is important.
- **Industrial Power Systems:** Serving motor loads and equipment in factories.

Mathematical Modeling The Y parameters (admittance matrix) relate the phase currents and voltages at the transformer terminals. Mutual and cross-phase couplings, as well as phase shift factors, are essential for a complete model. It must be noted here that the subscript p in this report describes the primary side of the transformer while the subscript s denotes the secondary side of the transformer. Moreover, it is also assumed that the three-phase transformer is a balanced one i.e. it has equal resistance and reactances for the three phases.

Admittance Calculation For each winding, the admittance Y is given by:

$$Y = \frac{1}{R + jX}$$

where R is the resistance, and X is the reactance. Each phase admittance on the primary and secondary sides is represented as:

$$Y_{p_i} = \frac{1}{R_{p_i} + jX_{p_i}}, \quad Y_{s_i} = \frac{1}{R_{s_i} + jX_{s_i}} \quad \text{for } i = 1, 2, 3$$

Y_{p_i} and Y_{s_i} represent the admittance of the primary (delta) and secondary (Y) windings, respectively.

Mutual Coupling Between Phases In a realistic three-phase transformer, there are mutual couplings between phases. Let M_p and M_s represent the mutual inductances between the primary and secondary phases. This mutual coupling is expressed as cross terms in the Y matrix.

Constructing the Y Matrix The Y matrix for a three-phase $\Delta - Y$ transformer can be generalized as a 6x6 matrix that includes mutual and cross-couplings between the phases along with the phase shift factors:

$$Y = \begin{bmatrix} Y_{p1} + M_{p12} & -M_{p12} & 0 & -aY_{p1}e^{-j\phi} & 0 & 0 \\ -M_{p12} & Y_{p2} + M_{p23} & -M_{p23} & 0 & -aY_{p2}e^{-j(\phi+\frac{2\pi}{3})} & 0 \\ 0 & -M_{p23} & Y_{p3} + M_{p31} & 0 & 0 & -aY_{p3}e^{-j(\phi-\frac{2\pi}{3})} \\ -aY_{p1}e^{j\phi} & 0 & 0 & Y_{s1} + M_{s12} & -M_{s12} & 0 \\ 0 & -aY_{p2}e^{j(\phi+\frac{2\pi}{3})} & 0 & -M_{s12} & Y_{s2} + M_{s23} & -M_{s23} \\ 0 & 0 & -aY_{p3}e^{j(\phi-\frac{2\pi}{3})} & 0 & -M_{s23} & Y_{s3} + M_{s31} \end{bmatrix}$$

where:

- Y_{p1}, Y_{p2}, Y_{p3} are the primary winding admittances.
- Y_{s1}, Y_{s2}, Y_{s3} are the secondary winding admittances.
- $M_{p12}, M_{p23}, M_{p31}, M_{s12}, M_{s23}, M_{s31}$ are the mutual couplings between different phases.
- a is the turns ratio.
- $e^{-j\phi}$ is the phase shift factor, with ϕ representing the phase angle difference between the delta and Y windings.

Voltage and Current Relationships with Phase Shift The voltage and current relationships in the transformer, with the phase shift ϕ between the primary and secondary windings, are as follows.

Voltage Relationships The primary side phase voltages for the delta configuration are:

$$V_{p1} = V_{ab}, \quad V_{p2} = V_{bc}, \quad V_{p3} = V_{ca}$$

where the subscript p refers to the primary side and the subscript s refers to the secondary side. V_{ab}, V_{bc}, V_{ca} refer the three-phase balanced voltages. The secondary side (Y) voltages are related by the turns ratio and the phase shift factor:

$$V_{s1} = \frac{V_{p1}}{a}e^{-j\phi}, \quad V_{s2} = \frac{V_{p2}}{a}e^{-j(\phi+\frac{2\pi}{3})}, \quad V_{s3} = \frac{V_{p3}}{a}e^{-j(\phi-\frac{2\pi}{3})}$$

Current Relationships On the delta (primary) side, the line currents are:

$$I_{p1} = I_{ab}, \quad I_{p2} = I_{bc}, \quad I_{p3} = I_{ca}$$

I_{ab}, I_{bc}, I_{ca} refer the three-phase balanced currents. The secondary side (Y) currents are:

$$I_{s1} = aI_{p1}e^{j\phi}, \quad I_{s2} = aI_{p2}e^{j(\phi-\frac{2\pi}{3})}, \quad I_{s3} = aI_{p3}e^{j(\phi+\frac{2\pi}{3})}$$

This model is much more generalized than the three-phase Delta-Delta transformer model as it incorporates the mutual and the cross couplings between the phases and also incorporates the phase shift factor between the primary and the secondary windings.

Implementation in Code This report provides a comprehensive explanation of the C++ code implemented for modeling a three-phase $\Delta - Y$ transformer in the file `transformer_Delta_Y.cpp` and the `transformer_Delta_Y.h`. The transformer is modeled as an electrical element with specified parameters such as resistances, reactances, turns ratio, and phase lag. For this report, we have implemented a zero phase shift factor and also neglected the mutual and cross-couplings as they are relatively small while formulating the Y matrices in the code.

Code Structure

Header Guards and Inheritance The file begins with header guards to prevent multiple inclusions during compilation, followed by inheritance from the `Element` class. The `Element` base class is used to represent generic electrical components, while the `TransformerDeltaY` class builds on this to represent a specific transformer model.

```
#ifndef TRANSFORMER_DELTA_Y_H
#define TRANSFORMER_DELTA_Y_H

#include "Element.h"
```

Listing 25: Header Guards and Inheritance

Explanation: - The `#ifndef` and `#define` directives ensure the header file is included only once during compilation, avoiding potential redefinition errors. - The `Element.h` file is included as the `TransformerDeltaY` class inherits from `Element`, making it a specialized element in the overall electrical system.

Class Declaration The class `TransformerDeltaY` is derived from the `Element` base class. This specialized class models a transformer that has a delta connection on the primary side and a star connection on the secondary side.

```
class TransformerDeltaY : public Element {
```

Listing 26: Class Declaration

Constructor and Destructor The constructor initializes the transformer object by passing a symbol (used for identification), the number of pins (representing phases), and a vector of values that typically include resistances, reactances, turns ratio, and phase lag. The destructor ensures that resources are properly managed when the transformer object is destroyed.

```
public:
    // Constructor to initialize the Transformer with a given symbol, number of pins, and values (with phase lag
        )
    TransformerDeltaY(const std::string& symbol, int pins, const std::vector<double>& values);

    ~TransformerDeltaY() override;
```

Listing 27: Constructor and Destructor

Explanation: - `symbol`: A string representing the transformer, which serves as an identifier within the system. - `pins`: An integer representing the number of pins or connections in the transformer (one per phase). - `values`: A vector of `double` values containing the electrical properties of the transformer such as resistance, reactance, turns ratio, and phase lag. - The destructor `~TransformerDeltaY()` ensures that any resources allocated for the transformer are properly released when it goes out of scope.

Accessor Methods The class defines several accessor methods to retrieve the properties of the transformer. These methods provide a way to access the resistance, reactance, turns ratio, and phase lag for a particular winding.

```
    double getResistance(int winding) const {
        if (winding >= 0 && winding < R.size()) {
            return R[winding];
        }
        throw std::out_of_range("Invalid winding index");
    }

    double getReactance(int winding) const {
```

```
        if (winding >= 0 && winding < X.size()) {
            return X[winding];
        }
        throw std::out_of_range("Invalid winding index");
    }

    double getTurnsRatio() const { return a; }

    double getPhaseLag() const { return phaseLag; } // Method to get phase lag
```

Listing 28: Accessor Methods

Explanation: - `getResistance(int winding)`: Returns the resistance for the specified winding (primary or secondary). It checks whether the index `winding` is within valid bounds, throwing an `out_of_range` exception if not. - `getReactance(int winding)`: Similar to the `getResistance()` method, this function returns the reactance of the specified winding, with the same bounds-checking mechanism. - `getTurnsRatio()`: This method returns the turns ratio a of the transformer, which is a key parameter for voltage transformation between the primary (delta) and secondary (Y) windings. - `getPhaseLag()`: This function returns the phase lag of the transformer, which represents the shift in phase between the primary and secondary sides of the transformer.

Private Member Variables The private section of the class contains member variables that store the resistances, reactances, turns ratio, and phase lag for the transformer. These variables are used internally by the class methods.

```
private:
    std::vector<double> R; // Resistances for primary (Delta side) and secondary (Star side) windings
    std::vector<double> X; // Reactances for primary and secondary windings
    double a; // Turns ratio
    double phaseLag; // Phase lag to incorporate in the Y parameters
```

Listing 29: Private Member Variables

Explanation: - `R`: A vector of resistances for the primary and secondary windings. The first portion of the vector corresponds to the resistances of the primary (delta) side, and the second portion corresponds to the secondary (star) side. - `X`: A vector of reactances that works similarly to `R`, with the first half for the primary side and the second half for the secondary side. - `a`: The turns ratio, which determines the voltage transformation between the delta and Y sides. - `phaseLag`: The phase lag accounts for any phase shifts between the primary and secondary windings, crucial for determining the transformer's Y-parameters in electrical network analysis.

Furthermore, this report also gives a detailed explanation of the `transformer_Delta_Y.cpp` file which can be explained as follows:

Code Structure: TransformerDeltaY.cpp The code listing below shows the implementation of the Delta-Y transformer:

```
#include "Transformer_Delta_Y.h"

// Constructor
TransformerDeltaY::TransformerDeltaY(const std::string& symbol, int pins, const std::vector<double>& values)
    : Element(symbol, pins, pins) {

    if (values.size() == 6) {
        R = { values[0], values[2] }; // Primary and secondary resistances
        X = { values[1], values[3] }; // Primary and secondary reactances
        a = values[4]; // Turns ratio
        phaseLag = values[5]; // Phase lag
    }
    else {
```

```
        throw std::invalid_argument("Invalid number of values, must be 6 (R_primary, X_primary, R_secondary, X_secondary, a, phaseLag)!");
    }

    // Check if the values are properly initialized
    for (int i = 0; i < 2; ++i) {
        if (R[i] == 0 || X[i] == 0 || a == 0) {
            std::cerr << "Transformer parameters not initialized correctly for winding " << i + 1 << "!" << std::endl;
            return;
        }
    }

    // Y parameters for the grounded star-delta three-phase transformer
    // We use a 6x6 Y matrix to represent the admittance between the 3 primary delta windings and 3 secondary star windings.

    RCP<const Basic> R_delta = real_double(R[0]);
    RCP<const Basic> X_delta = real_double(X[0]);
    RCP<const Basic> R_star = real_double(R[1]);
    RCP<const Basic> X_star = real_double(X[1]);
    RCP<const Basic> a_val = real_double(a); // Turns ratio symbol

    RCP<const Basic> phaseFactor = exp(mul(neg(j), real_double(phaseLag))); // Phase shift

    // Compute primary (delta) and secondary (star) admittances
    RCP<const Basic> Y_delta = div(real_double(1), add(R_delta, mul(j, X_delta)));
    RCP<const Basic> Y_star = div(real_double(1), add(R_star, mul(j, X_star)));

    // Define the 3x3 submatrices for the delta and star connections
    DenseMatrix Y_delta_matrix(3, 3);
    DenseMatrix Y_star_matrix(3, 3);

    // Populate delta side Y matrix (for primary side)
    Y_delta_matrix.set(0, 0, Y_delta); // Y11
    Y_delta_matrix.set(0, 1, mul(real_double(-1), Y_delta)); // Y12
    Y_delta_matrix.set(0, 2, real_double(0)); // No direct interaction between phase A and C

    Y_delta_matrix.set(1, 0, mul(real_double(-1), Y_delta)); // Y21
    Y_delta_matrix.set(1, 1, Y_delta); // Y22
    Y_delta_matrix.set(1, 2, mul(real_double(-1), Y_delta)); // Y23

    Y_delta_matrix.set(2, 0, real_double(0)); // No direct interaction between phase C and A
    Y_delta_matrix.set(2, 1, mul(real_double(-1), Y_delta)); // Y32
    Y_delta_matrix.set(2, 2, Y_delta); // Y33

    // Populate star side Y matrix (for secondary side)
    Y_star_matrix.set(0, 0, Y_star); // Y11
    Y_star_matrix.set(0, 1, real_double(0)); // No direct interaction between phase A and B
    Y_star_matrix.set(0, 2, real_double(0)); // No direct interaction between phase A and C

    Y_star_matrix.set(1, 0, real_double(0)); // No direct interaction between phase B and A
    Y_star_matrix.set(1, 1, Y_star); // Y22
    Y_star_matrix.set(1, 2, real_double(0)); // No direct interaction between phase B and C

    Y_star_matrix.set(2, 0, real_double(0)); // No direct interaction between phase C and A
    Y_star_matrix.set(2, 1, real_double(0)); // No direct interaction between phase C and B
    Y_star_matrix.set(2, 2, Y_star); // Y33

    // Assign submatrices to Y_matrix (6x6 for 3 primary and 3 secondary)
    // Top-left 3x3: Y_delta (primary winding)
    for (int i = 0; i < 3; ++i) {
        for (int j = 0; j < 3; ++j) {
            Y_matrix.set(i, j, Y_delta_matrix.get(i, j)); // Y_delta elements
            Y_matrix.set(i + 3, j + 3, Y_star_matrix.get(i, j)); // Y_star elements
            Y_matrix.set(i, j + 3, mul(real_double(-a), Y_delta_matrix.get(i, j))); // Y12 interaction (primary to secondary)
            Y_matrix.set(i + 3, j, mul(real_double(-1), Y_delta_matrix.get(i, j))); // Y21 interaction (
```

```
                secondary to primary)
        }
    }
}

// Destructor
TransformerDeltaY::~TransformerDeltaY() {
    std::cout << "Transformer Delta Y object for " << getElementSymbol() << " destroyed." << std::endl;
}
```

Listing 30: TransformerDeltaY.cpp

Explanation of the Code

Constructor Initialization The constructor `TransformerDeltaY` initializes the transformer by accepting a symbolic name, the number of pins, and the list of transformer parameters, which include:

- Primary and secondary resistances (`R`).
- Primary and secondary reactances (`X`).
- Turns ratio (`a`).
- Phase lag between the delta and star windings (`phaseLag`).

These parameters are stored in vectors to represent the delta (primary) and star (secondary) side components. If the parameters are not initialized properly, the constructor throws an error.

Admittance Matrix Calculation The admittance (Y) matrix is a 6x6 matrix that models the relationship between the voltages and currents across the transformer terminals. This matrix is composed of submatrices:

- `Y_delta_matrix`: The 3x3 admittance matrix for the delta side (primary windings).
- `Y_star_matrix`: The 3x3 admittance matrix for the star side (secondary windings).
- `Y_12`: Interaction matrix representing the coupling from delta to star side.
- `Y_21`: Interaction matrix representing the coupling from star to delta side.

Delta Side Admittance The delta side admittance (`Y_delta`) is calculated using the primary resistance and reactance. The admittance formula is:

$$Y_\Delta = \frac{1}{R_\Delta + jX_\Delta}$$

This is then populated into the 3x3 `Y_delta_matrix`, where negative off-diagonal values represent mutual interactions between phases.

Star Side Admittance Similarly, the star side admittance (`Y_star`) is calculated using the secondary resistance and reactance:

$$Y_Y = \frac{1}{R_Y + jX_Y}$$

This is used to populate the 3x3 `Y_star_matrix`, with diagonal elements representing self-admittance for each phase.

Interaction Between Delta and Star Sides The interaction matrices between the delta and star sides (`Y_12` and `Y_21`) are scaled by the turns ratio `a` and represent the coupling between the primary and secondary windings. These elements are placed in the appropriate positions of the 6x6 matrix.

Destructor Definition The destructor is defined to clean up resources when an object of the transformer class is destroyed:

```
TransformerDeltaY::~TransformerDeltaY() {
    std::cout << "Transformer Delta Y object for " << getElementSymbol() << " destroyed." << std::endl;
}
```

This function outputs a message indicating that the transformer object has been destroyed.

Introduction The Star-Delta ($Y-\Delta$) transformer is a fundamental component in power systems, serving to convert voltage levels between the star (Y) configuration and the delta (Δ) configuration. The star side is typically grounded, which helps stabilize the system and manage unbalanced loads. This transformer configuration is particularly beneficial for applications where isolation, voltage transformation, and reduced phase voltage are required.

This report focuses on the mathematical model of a generalized Star-Delta transformer, incorporating the phase shift factors and the mutual and the cross-coupling between the phases as shown in the three-phase $\Delta - Y$ transformer model.

Applications The Star-Delta configuration finds application in various domains, including:

- **Power Distribution:** Transforming voltages for distribution to various loads.
- **Motor Start Applications:** Providing reduced starting current for induction motors, which can lead to a smoother start.
- **Harmonic Mitigation:** Managing harmonics in the electrical network through proper winding connections.

Mathematical Modeling The Y parameters (admittance matrix) are crucial for defining the relationships between phase currents and voltages at the transformer terminals. A comprehensive model includes mutual and cross-phase couplings, alongside phase shift factors.

Admittance Calculation The admittance Y for each winding can be expressed as:

$$Y = \frac{1}{R + jX}$$

where R represents the resistance, and X denotes the reactance. The phase admittance on the primary (Y) and secondary (Δ) sides can be represented as:

$$Y_{s_i} = \frac{1}{R_{s_i} + jX_{s_i}}, \quad Y_{p_i} = \frac{1}{R_{p_i} + jX_{p_i}} \quad \text{for } i = 1, 2, 3$$

where Y_{s_i} and Y_{p_i} denote the admittance of the star (Y) and delta (Δ) windings, respectively.

Mutual Coupling Between Phases To reflect the realistic behavior of a Star-Delta transformer, mutual couplings between phases must be included. Let M_s and M_p represent the mutual inductances between the star and delta phases, respectively. This mutual coupling is represented as cross terms in the Y matrix.

Constructing the Y Matrix The Y matrix for a three-phase star-delta transformer can be expressed as a 6×6 matrix, encompassing mutual and cross-couplings:

$$Y = \begin{bmatrix} Y_{s1} + M_{s12} & -M_{s12} & -M_{s13} & -aY_{p1}e^{-j\phi} & 0 & 0 \\ -M_{s12} & Y_{s2} + M_{s23} & -M_{s23} & 0 & -aY_{p2}e^{-j(\phi+\frac{2\pi}{3})} & 0 \\ -M_{s13} & -M_{s23} & Y_{s3} + M_{s31} & 0 & 0 & -aY_{p3}e^{-j(\phi-\frac{2\pi}{3})} \\ -aY_{p1}e^{j\phi} & 0 & 0 & Y_{p1} + M_{p12} & -M_{p12} & -M_{p13} \\ 0 & -aY_{p2}e^{j(\phi-\frac{2\pi}{3})} & 0 & -M_{p12} & Y_{p2} + M_{p23} & -M_{p23} \\ 0 & 0 & -aY_{p3}e^{j(\phi+\frac{2\pi}{3})} & -M_{p13} & -M_{p23} & Y_{p3} + M_{p31} \end{bmatrix}$$

where:

- Y_{s1}, Y_{s2}, Y_{s3} are the admittances for the star winding.
- Y_{p1}, Y_{p2}, Y_{p3} are the admittances for the delta winding.
- $M_{s12}, M_{s13}, M_{s23}, M_{p12}, M_{p13}, M_{p23}$ are the mutual couplings between different phases.
- a is the turns ratio.
- $e^{-j\phi}$ represents the phase shift factor, where ϕ indicates the phase angle difference between the star and delta windings.

Voltage and Current Relationships with Phase Shift The voltage and current relationships in the transformer, integrating the phase shift ϕ between the star and delta windings, are outlined below.

Voltage Relationships The primary side phase voltages for the star configuration are:

$$V_{s1} = V_{aN}, \quad V_{s2} = V_{bN}, \quad V_{s3} = V_{cN}$$

where V_{aN}, V_{bN}, V_{cN} represent the balanced three-phase voltages with respect to ground. The secondary side (delta) voltages relate to the primary side through the turns ratio and the phase shift factor:

$$V_{p1} = aV_{s1}e^{j\phi}, \quad V_{p2} = aV_{s2}e^{j(\phi-\frac{2\pi}{3})}, \quad V_{p3} = aV_{s3}e^{j(\phi+\frac{2\pi}{3})}$$

Current Relationships On the star (primary) side, the three-phase balanced line currents (I_a, I_b, I_c) are:

$$I_{s1} = I_a, \quad I_{s2} = I_b, \quad I_{s3} = I_c$$

The secondary side (delta) currents can be expressed as:

$$I_{p1} = \frac{I_{s1}}{a}e^{-j\phi}, \quad I_{p2} = \frac{I_{s2}}{a}e^{-j(\phi-\frac{2\pi}{3})}, \quad I_{p3} = \frac{I_{s3}}{a}e^{-j(\phi+\frac{2\pi}{3})}$$

This model provides a detailed representation of the Star-Delta transformer, capturing the essential interactions and relationships between the phases while accounting for phase shifts and mutual coupling effects.

Implementation in Code This report provides a comprehensive explanation of the C++ code implemented for modelling a three-phase $Y-\Delta$ transformer in the file `transformer_Y_Delta.cpp` and the `transformer_Y_Delta.h`. The transformer is modelled as an electrical element with specified parameters such as resistances, reactances, turns ratio, and phase lag. Similar to the three-phase $\Delta - Y$ transformer code implementation, for simplicity we have neglected the mutual and cross-couplings as they are relatively small while formulating the Y matrices and also assumed a zero phase shift factor as shown in this report.

This report explains the code structure for the implementation of a three-phase Star-Delta transformer model. The model is written in C++ and includes a header file defining the structure of the transformer class, with functions for accessing parameters such as resistances, reactances, turns ratio, and phase lag. This class is crucial for simulating the electrical behavior of a Star-Delta transformer in a power system.

Code Listing: TransformerYDelta.h Below is the C++ code listing for the three-phase Star-Delta transformer class definition:

```
#ifndef TRANSFORMER_Y_DELTA_H
#define TRANSFORMER_Y_DELTA_H

#include "Element.h"

class TransformerYDelta : public Element {
public:
```

```
    // Constructor to initialize the Transformer with a given symbol, number of pins, and values (including
        phase lag)
    TransformerYDelta(const std::string& symbol, int pins, const std::vector<double>& values);

    ~TransformerYDelta() override;

    double getResistance(int winding) const {
        if (winding >= 0 && winding < R.size()) {
            return R[winding];
        }
        throw std::out_of_range("Invalid winding index");
    }

    double getReactance(int winding) const {
        if (winding >= 0 && winding < X.size()) {
            return X[winding];
        }
        throw std::out_of_range("Invalid winding index");
    }

    double getTurnsRatio() const { return a; }

    double getPhaseLag() const { return phaseLag; } // Method to retrieve phase lag

private:
    std::vector<double> R; // Resistances for primary (Y side) and secondary (Delta side) windings
    std::vector<double> X; // Reactances for primary and secondary windings
    double a; // Turns ratio
    double phaseLag; // Phase lag to incorporate in Y parameters
};

#endif // TRANSFORMER_Y_DELTA_H
```

Listing 31: TransformerYDelta.h

Explanation of Code Structure

Class Declaration The class `TransformerYDelta` inherits from the base class `Element`, which represents electrical components in the simulation framework. The constructor initializes the transformer with essential parameters such as resistances, reactances, turns ratio, and phase lag. These parameters are used to compute the transformer's electrical behavior when simulating power systems.

Public Methods

- `TransformerYDelta(const std::string& symbol, int pins, const std::vector<double>& values):`
  This constructor initializes a transformer with a given symbol (e.g., transformer name), the number of pins (6 in the case of a three-phase transformer), and a list of parameters (`values`) containing resistances, reactances, turns ratio, and phase lag. This ensures that all necessary electrical properties are initialized at object creation.

- `~TransformerYDelta() override:`
  The destructor is defined to safely destroy any instances of `TransformerYDelta`. This is marked as `override` to emphasize that it overrides the virtual destructor of the base `Element` class.

- `double getResistance(int winding) const:`
  This method returns the resistance of the specified winding. There are two windings (primary for the star side and secondary for the delta side). The method checks if the winding index is valid (either 0 or 1) and returns the respective resistance value. If an invalid index is provided, it throws an `out_of_range` exception.

- `double getReactance(int winding) const:`
  This method returns the reactance of the specified winding. Like the `getResistance` method, it checks if the index is valid and retrieves the corresponding reactance for the specified winding.

- `double getTurnsRatio() const:`
  This method simply returns the turns ratio (`a`) of the transformer. The turns ratio defines the relationship between the primary and secondary winding voltages and currents.

- `double getPhaseLag() const:`
  This method returns the phase lag (`phaseLag`) between the primary and secondary sides of the transformer. The phase lag is used in power system simulations to account for phase shifts that occur due to transformer winding configurations.

Private Data Members The private members store the parameters of the transformer:

- `std::vector<double> R;`
  This vector stores the resistances for both the primary and secondary windings. The primary winding (Y side) and secondary winding (Delta side) have their respective resistance values.

- `std::vector<double> X;`
  This vector stores the reactances for the primary and secondary windings. The primary and secondary reactances play a significant role in determining the impedance of the transformer windings.

- `double a;`
  The turns ratio (`a`) is a critical parameter that defines the ratio between the primary (Y side) and secondary (Delta side) winding voltages. It is also used to scale the secondary current based on the primary current.

- `double phaseLag;`
  The phase lag represents the shift in phase angle between the Y and Delta sides of the transformer. This is an important characteristic for power systems involving star-delta transformers, where phase shifts occur due to the different winding configurations.

Key Features of the Star-Delta Transformer A Star-Delta transformer configuration is widely used in power transmission and distribution systems. Some of its key features are:

- **Phase Shift**: The transformer introduces a phase shift between the primary and secondary windings due to the star and delta configurations. This phase shift is captured by the `phaseLag` parameter.

- **Turns Ratio**: The turns ratio (`a`) defines the relationship between the primary and secondary voltages and currents. For example, if the turns ratio is 2:1, the primary voltage will be twice that of the secondary.

- **Impedance Representation**: The resistances (`R`) and reactances (`X`) represent the impedance characteristics of the windings, which are important for determining how the transformer responds to different electrical loads.

- **Grounding on Primary Side**: In a star-delta configuration, the primary side (Y side) is typically grounded, which helps to stabilize the voltage and reduce the likelihood of voltage imbalances in the system.

The `TransformerYDelta` class provides a comprehensive implementation of a three-phase star-delta transformer. The class offers methods to access and modify important transformer parameters such as resistance, reactance, turns ratio, and phase lag. These attributes are essential for simulating the behaviour of the transformer in power systems. By encapsulating these parameters in a C++ class, it becomes easier to integrate the transformer into larger simulations and perform detailed analyses of its performance under various operating conditions.

Introduction This report provides a detailed explanation of the C++ implementation of a three-phase Y-Delta transformer. The transformer model calculates the admittance matrix for both the primary (Y) and secondary (Delta) windings, based on provided electrical parameters such as resistance, reactance, turns ratio, and phase lag. The code structure and function are discussed, and each section of the code is explained in detail.

Code Structure The 'TransformerYDelta' class in the file 'transformer_Y_Delta.cpp' defines the behaviour of the transformer. This class inherits from the base class 'Element'. The key responsibilities of this class are to:

- Initialize the transformer's electrical parameters.
- Compute the Y matrix for the transformer, which consists of 6x6 elements representing the admittance relationships between the 3 primary and 3 secondary windings.
- Ensure proper resource management with a destructor.

Code Explanation

Constructor and Initialization The constructor initializes the transformer with the given symbol, number of pins, and electrical parameters, including resistances, reactances, turns ratio, and phase lag.

```
TransformerYDelta::TransformerYDelta(const std::string& symbol, int pins, const std::vector<double>& values)
    : Element(symbol, pins, pins) {

    if (values.size() == 6) {
        R = { values[0], values[2] }; // Primary (Y side) and secondary (Delta side) resistances
        X = { values[1], values[3] }; // Primary (Y side) and secondary (Delta side) reactances
        a = values[4]; // Turns ratio
        phaseLag = values[5]; // Phase lag
    }
    else {
        throw std::invalid_argument("Invalid number of values, must be 6 (R_primary, X_primary, R_secondary,
            X_secondary, a, phaseLag)!");
    }
```

Listing 32: Constructor of the TransformerYDelta class

The constructor first checks if the number of input values is correct (six values representing the resistance, reactance, turns ratio, and phase lag). If the values are correct, it assigns the resistance and reactance values for the primary (Y) and secondary (Delta) sides, as well as the turns ratio and phase lag. If the number of values is incorrect, an exception is thrown.

Parameter Validation The constructor includes a validation loop to check whether the transformer parameters have been initialized properly. If any of the resistances, reactances, or the turns ratio are zero, an error message is printed.

```
    // Check if the values are properly initialized
    for (int i = 0; i < 2; ++i) {
        if (R[i] == 0 || X[i] == 0 || a == 0) {
            std::cerr << "Transformer parameters not initialized correctly for winding " << i + 1 << "!" << std
                ::endl;
            return;
        }
    }
```

Listing 33: Parameter validation for the transformer windings

This ensures that the transformer is configured correctly before proceeding with the admittance matrix computation.

Y-Parameters Calculation The Y (admittance) matrix is calculated using the primary and secondary resistances and reactances. The admittances are computed for both the primary and secondary windings based on the formula:

$$Y = \frac{1}{R + jX}$$

where R is the resistance, X is the reactance, and j is the imaginary unit.

```
RCP<const Basic> R_p = real_double(R[0]); // Primary (Y side) resistance
RCP<const Basic> X_p = real_double(X[0]); // Primary (Y side) reactance
RCP<const Basic> R_s = real_double(R[1]); // Secondary (Delta side) resistance
RCP<const Basic> X_s = real_double(X[1]); // Secondary (Delta side) reactance
RCP<const Basic> a_val = real_double(a); // Turns ratio

// Compute primary and secondary admittances
RCP<const Basic> Y_p = div(real_double(1), add(R_p, mul(j, X_p))); // Admittance for Y side
RCP<const Basic> Y_s = div(real_double(1), add(R_s, mul(j, X_s))); // Admittance for Delta side
```

Listing 34: Computation of admittances for Y and Delta windings

Here, the primary and secondary admittances are calculated as Y_p and Y_s, respectively. These admittances are later used to construct the Y matrix for the transformer.

Y Matrix Construction The 6x6 Y matrix represents the relationship between the 3 primary windings (Y side) and the 3 secondary windings (Delta side). The matrix is populated based on the computed admittances.

```
DenseMatrix Y_y(3, 3); // Primary Y side
DenseMatrix Y_delta(3, 3); // Secondary Delta side

// Populate Y matrix for primary (Y side)
Y_y.set(0, 0, Y_p); // Y11
Y_y.set(0, 1, mul(real_double(-1), Y_p)); // Y12
Y_y.set(0, 2, real_double(0)); // No direct interaction between phase A and C
```

Listing 35: Population of the Y matrix

This snippet shows the population of the Y matrix for the primary side (Y configuration). The admittance values are set for each phase interaction (e.g., between phases A, B, and C). The same process is repeated for the secondary Delta side.

```
Y_delta.set(0, 0, Y_s); // Y11
Y_delta.set(0, 1, mul(real_double(-1), Y_s)); // Y12
Y_delta.set(0, 2, real_double(0)); // No direct interaction between phase A and C
```

Listing 36: Population of the secondary Delta side of the Y matrix

The same logic is applied to the Delta side, with the respective admittances set for each phase interaction.

Combining the Matrices The 6x6 Y matrix is completed by combining the primary and secondary submatrices. Interactions between the primary and secondary windings are also taken into account, with the turns ratio being used to scale the admittances.

```
for (int i = 0; i < 3; ++i) {
    for (int j = 0; j < 3; ++j) {
        Y_matrix.set(i, j, Y_y.get(i, j)); // Primary Y elements
        Y_matrix.set(i + 3, j + 3, Y_delta.get(i, j)); // Secondary Delta elements
        Y_matrix.set(i, j + 3, mul(real_double(-a), Y_y.get(i, j))); // Y12 interaction
        Y_matrix.set(i + 3, j, mul(real_double(-1), Y_y.get(i, j))); // Y21 interaction
    }
}
```

Listing 37: Combining the Y matrix for primary and secondary windings

This section fills in the remaining elements of the Y matrix, accounting for the interactions between the primary and secondary windings.

Destructor The destructor is responsible for cleaning up resources when the 'TransformerYDelta' object is destroyed.

```
TransformerYDelta::~TransformerYDelta() {
    std::cout << "TransformerYDelta object for " << getElementSymbol() << " destroyed." << std::endl;
}
```

Listing 38: Destructor of the TransformerYDelta class

This destructor simply prints a message indicating that the object has been destroyed.

Introduction The Star-Star ($Y-Y$) transformer is a vital component in power systems, primarily used for voltage level conversion between the star (Y) configuration. The star sides are typically grounded, providing system stability and facilitating the management of unbalanced loads. This transformer configuration is particularly beneficial for applications requiring voltage transformation, improved system reliability, and effective load balancing.

This report focuses on the mathematical model of a generalised Star-Star transformer, incorporating the phase shift factors and the mutual and the cross-couplings between the phases as shown in the $Y-\Delta$ three-phase transformer model.

Applications The Star-Star configuration finds applications in various domains, including:

- **Power Distribution:** Transforming voltages for efficient distribution to various loads.
- **Load Balancing:** Managing phase imbalances in power systems to enhance overall system performance.
- **Harmonic Mitigation:** Reducing harmonics in the electrical network through proper winding connections.

Mathematical Modeling The Y parameters (admittance matrix) are crucial for defining the relationships between phase currents and voltages at the transformer terminals. A comprehensive model includes mutual and cross-phase couplings, alongside phase shift factors.

Admittance Calculation The admittance Y for each winding can be expressed as:

$$Y = \frac{1}{R + jX}$$

where R represents the resistance, and X denotes the reactance. The phase admittance on the primary (Y) and secondary (Y) sides can be represented as:

$$Y_{s_i} = \frac{1}{R_{s_i} + jX_{s_i}}, \quad Y_{p_i} = \frac{1}{R_{p_i} + jX_{p_i}} \quad \text{for } i = 1, 2, 3$$

where Y_{s_i} and Y_{p_i} denote the admittance of the star (Y) windings.

Mutual Coupling Between Phases To reflect the realistic behavior of a Star-Star transformer, mutual couplings between phases must be included. Let M_s represent the mutual inductance between the star phases. This mutual coupling is represented as cross terms in the Y matrix.

Constructing the Y Matrix The Y matrix for a three-phase Star-Star transformer can be expressed as a 6×6 matrix, encompassing mutual and cross-couplings:

$$Y = \begin{bmatrix} Y_{s1} + M_{s12} & -M_{s12} & -M_{s13} & -aY_{p1}e^{-j\phi} & 0 & 0 \\ -M_{s12} & Y_{s2} + M_{s23} & -M_{s23} & 0 & -aY_{p2}e^{-j(\phi+\frac{2\pi}{3})} & 0 \\ -M_{s13} & -M_{s23} & Y_{s3} + M_{s31} & 0 & 0 & -aY_{p3}e^{-j(\phi-\frac{2\pi}{3})} \\ -aY_{p1}e^{j\phi} & 0 & 0 & Y_{p1} + M_{p12} & -M_{p12} & -M_{p13} \\ 0 & -aY_{p2}e^{j(\phi-\frac{2\pi}{3})} & 0 & -M_{p12} & Y_{p2} + M_{p23} & -M_{p23} \\ 0 & 0 & -aY_{p3}e^{j(\phi+\frac{2\pi}{3})} & -M_{p13} & -M_{p23} & Y_{p3} + M_{p31} \end{bmatrix}$$

where:

- Y_{s1}, Y_{s2}, Y_{s3} are the admittances for the star winding.
- Y_{p1}, Y_{p2}, Y_{p3} are the admittances for the secondary winding.
- $M_{s12}, M_{s13}, M_{s23}, M_{p12}, M_{p13}, M_{p23}$ are the mutual couplings between different phases.
- a is the turns ratio.
- $e^{-j\phi}$ represents the phase shift factor, where ϕ indicates the phase angle difference between the windings.

Voltage and Current Relationships with Phase Shift The voltage and current relationships in the transformer, integrating the phase shift ϕ between the windings, are outlined below.

Voltage Relationships The primary side phase voltages for the star configuration are:

$$V_{s1} = V_{aN}, \quad V_{s2} = V_{bN}, \quad V_{s3} = V_{cN}$$

where V_{aN}, V_{bN}, V_{cN} represent the balanced three-phase voltages with respect to ground. The secondary side (star) voltages relate to the primary side through the turns ratio and the phase shift factor:

$$V_{p1} = aV_{s1}e^{j\phi}, \quad V_{p2} = aV_{s2}e^{j(\phi-\frac{2\pi}{3})}, \quad V_{p3} = aV_{s3}e^{j(\phi+\frac{2\pi}{3})}$$

Current Relationships On the star (primary) side, the line currents (I_a, I_b, I_c) are:

$$I_{s1} = I_a, \quad I_{s2} = I_b, \quad I_{s3} = I_c$$

The secondary side (star) currents can be expressed as:

$$I_{p1} = \frac{I_{s1}}{a}e^{-j\phi}, \quad I_{p2} = \frac{I_{s2}}{a}e^{-j(\phi-\frac{2\pi}{3})}, \quad I_{p3} = \frac{I_{s3}}{a}e^{-j(\phi+\frac{2\pi}{3})}$$

This model provides a detailed representation of the Star-Star transformer, capturing the essential interactions and relationships between the phases while accounting for phase shifts and mutual coupling effects.

Introduction This report provides a comprehensive explanation of the C++ code implemented for modelling a three-phase $Y-Y$ transformer in the file `transformer_Y_Y.cpp` and the `transformer_Y_Y.h`. The transformer is modelled as an electrical element with specified parameters such as resistances, reactances, turns ratio, and phase lag. Similar to the three-phase $\Delta - Y$ transformer code implementation, for simplicity we have neglected the mutual and cross-couplings as they are relatively small while formulating the Y matrices and also assumed a zero phase shift factor as shown in this report.

Header File Structure The header file for the `TransformerYY` class is as follows:

```
#pragma once
#ifndef TRANSFORMER_Y_Y_H
#define TRANSFORMER_Y_Y_H

#include "Element.h"

class TransformerYY : public Element {
public:
    TransformerYY(const std::string& symbol, int pins, const std::vector<double>& values);
    ~TransformerYY() override;

    double getResistance(int winding) const;
    double getReactance(int winding) const;
    double getTurnsRatio() const;
```

```
    double getPhaseLag() const;

private:
    std::vector<double> R; // Resistances for primary and secondary windings
    std::vector<double> X; // Reactances for primary and secondary windings
    double a; // Turns ratio
    double phaseLag; // Phase lag to incorporate in the Y parameters
};

#endif // TRANSFORMER_Y_Y_H
```

Overview The `TransformerYY` class inherits from the `Element` class, representing a three-phase transformer with both the primary and secondary windings connected in a Y (star) configuration. This class provides methods to initialize the transformer, retrieve its electrical parameters, and handle specific functionalities related to its operation.

Code Explanation

Header Guards The file begins with header guards to prevent multiple inclusions:

```
#pragma once
#ifndef TRANSFORMER_Y_Y_H
#define TRANSFORMER_Y_Y_H
```

The `#pragma once` directive and the traditional header guards ensure that this header file is included only once in a compilation unit.

Class Declaration The class is declared as follows:

```
#include "Element.h"

class TransformerYY : public Element {
```

This declaration indicates that `TransformerYY` inherits from the `Element` class, allowing it to utilize properties and methods defined in `Element`.

Public Methods and Constructor The public methods and constructor are defined next:

```
public:
    TransformerYY(const std::string& symbol, int pins, const std::vector<double>& values);
    ~TransformerYY() override;

    double getResistance(int winding) const;
    double getReactance(int winding) const;
    double getTurnsRatio() const;
    double getPhaseLag() const;
```

Constructor The constructor initializes the `TransformerYY` object:

```
TransformerYY(const std::string& symbol, int pins, const std::vector<double>& values);
```

It takes three parameters: a symbol for the transformer, the number of pins (terminals), and a vector of values representing the transformer's parameters (resistances, reactances, turns ratio, and phase lag). The constructor's implementation typically assigns these values to the class attributes.

Destructor The destructor is defined as follows:

```
~TransformerYY() override;
```

The destructor cleans up resources when a `TransformerYY` object is destroyed.

Get Methods The class contains several getter methods:

```
double getResistance(int winding) const;
double getReactance(int winding) const;
double getTurnsRatio() const;
double getPhaseLag() const;
```

- `getResistance(int winding)`: Retrieves the resistance of the specified winding. It checks if the winding index is valid and returns the corresponding resistance, throwing an exception if the index is invalid.
- `getReactance(int winding)`: Similar to `getResistance`, this method retrieves the reactance of the specified winding.
- `getTurnsRatio()`: Returns the turns ratio a of the transformer, indicating the ratio of the number of turns in the primary winding to the number of turns in the secondary winding.
- `getPhaseLag()`: Returns the phase lag associated with the transformer, relevant for calculating Y parameters.

Private Members The private members are defined as follows:

```
private:
    std::vector<double> R; // Resistances for primary and secondary windings
    std::vector<double> X; // Reactances for primary and secondary windings
    double a; // Turns ratio
    double phaseLag; // Phase lag to incorporate in the Y parameters
```

- `std::vector<double> R`: Stores the resistances for both primary and secondary windings.
- `std::vector<double> X`: Holds the reactances for both primary and secondary windings.
- `double a`: Represents the turns ratio of the transformer.
- `double phaseLag`: Stores the phase lag associated with the transformer.

The `TransformerYY` class provides a structured way to define and interact with a three-phase Y-Y transformer in a power system simulation or analysis application. It encapsulates important electrical parameters, allows for parameter retrieval, and ensures safe access to these parameters with appropriate bounds checking. The design promotes a clean separation of the transformer model's data and functionality, making it easier to integrate into larger systems.

Introduction This report provides a comprehensive overview of the implementation of the `TransformerYY` class in C++, which models a three-phase Y-Y (star-star) transformer. The class encapsulates the behavior and parameters of the transformer, including its resistances, reactances, turns ratio, and phase lag.

Source File Structure The source file for the `TransformerYY` class implementation is structured as follows:

```
#include "Transformer_Y_Y.h"
#include <cmath>

// Constructor
TransformerYY::TransformerYY(const std::string& symbol, int pins, const std::vector<double>& values)
    : Element(symbol, pins, pins) {
```

Overview The `TransformerYY.cpp` file includes the implementation of the constructor and destructor of the `TransformerYY` class, along with the initialization and calculation of the transformer parameters. The class is designed to handle both primary and secondary windings of the transformer.

Code Explanation

Include Statements The file begins with the following include statements:

```
#include "Transformer_Y_Y.h"
#include <cmath>
```

The first line includes the header file for the `TransformerYY` class, while the second line includes the mathematical functions from the C++ standard library.

Constructor Implementation The constructor of the `TransformerYY` class is defined as follows:

```
TransformerYY::TransformerYY(const std::string& symbol, int pins, const std::vector<double>& values)
    : Element(symbol, pins, pins) {
```

This constructor initializes the `TransformerYY` object by calling the constructor of the base class `Element` and setting the symbol and number of pins for the transformer.

Parameter Initialization The constructor checks if the input vector `values` contains exactly six elements:

```
if (values.size() == 6) {
    R = { values[0], values[2] }; // Primary and secondary resistances
    X = { values[1], values[3] }; // Primary and secondary reactances
    a = values[4]; // Turns ratio
    phaseLag = values[5]; // Phase lag
}
else {
    throw std::invalid_argument("Invalid number of values, must be 6 (R_primary, X_primary, R_secondary,
        X_secondary, a, phaseLag)!");
}
```

If the vector size is incorrect, it throws an exception indicating the required number of values. The resistances and reactances for both the primary and secondary windings are initialized based on the input values.

Parameter Validation The following loop checks if the parameters are correctly initialized:

```
for (int i = 0; i < 2; ++i) {
    if (R[i] == 0 || X[i] == 0 || a == 0) {
        std::cerr << "Transformer parameters not initialized correctly for winding " << i + 1 << "!" << std::
            endl;
        return;
    }
}
```

If any resistance, reactance, or turns ratio is zero, an error message is printed to the console.

Y Parameter Calculation The transformer uses a 6x6 Y matrix to represent the admittance between the three primary windings and three secondary windings. The code calculates the Y parameters as follows:

```
RCP<const Basic> R_primary = real_double(R[0]);
RCP<const Basic> X_primary = real_double(X[0]);
RCP<const Basic> R_secondary = real_double(R[1]);
RCP<const Basic> X_secondary = real_double(X[1]);
RCP<const Basic> a_val = real_double(a); // Turns ratio symbol

RCP<const Basic> phaseFactor = exp(mul(neg(j), real_double(phaseLag))); // Phase shift
```

Here, the resistances and reactances are converted into a symbolic representation. The phase factor for the phase lag is calculated using the exponential function.

Admittance Calculation The primary and secondary admittances are computed as follows:

```
RCP<const Basic> Y_primary = div(real_double(1), add(R_primary, mul(j, X_primary)));
RCP<const Basic> Y_secondary = div(real_double(1), add(R_secondary, mul(j, X_secondary)));
```

These formulas calculate the admittance by taking the inverse of the complex impedance (which is the sum of resistance and reactance).

Y Matrix Population The code defines two matrices for the primary and secondary windings:

```
DenseMatrix Y_primary_matrix(3, 3);
DenseMatrix Y_secondary_matrix(3, 3);
```

Next, the matrices are populated:

```
// Populate primary side Y matrix (for primary winding)
Y_primary_matrix.set(0, 0, Y_primary); // Y11
Y_primary_matrix.set(1, 1, Y_primary); // Y22
Y_primary_matrix.set(2, 2, Y_primary); // Y33

// Populate secondary side Y matrix (for secondary winding)
Y_secondary_matrix.set(0, 0, Y_secondary); // Y11
Y_secondary_matrix.set(1, 1, Y_secondary); // Y22
Y_secondary_matrix.set(2, 2, Y_secondary); // Y33
```

Each matrix is set to have the respective admittance values on the diagonal, reflecting the absence of direct interaction between the phases.

Final Y Matrix Assembly The following loop assembles the final 6x6 Y matrix that incorporates the interactions between primary and secondary windings:

```
for (int i = 0; i < 3; ++i) {
    for (int j = 0; j < 3; ++j) {
        Y_matrix.set(i, j, Y_primary_matrix.get(i, j)); // Y_primary elements
        Y_matrix.set(i + 3, j + 3, Y_secondary_matrix.get(i, j)); // Y_secondary elements
        Y_matrix.set(i, j + 3, mul(real_double(-a), Y_primary_matrix.get(i, j))); // Y12 interaction (primary
            to secondary)
        Y_matrix.set(i + 3, j, mul(real_double(-1), Y_primary_matrix.get(i, j))); // Y21 interaction (secondary
            to primary)
    }
}
```

This part of the code ensures that the Y matrix is correctly filled with the primary and secondary admittance values, as well as the interaction terms due to the turns ratio.

Destructor Implementation The destructor of the `TransformerYY` class is defined as follows:

```
// Destructor
TransformerYY::~TransformerYY() {
    std::cout << "TransformerYY object for " << getElementSymbol() << " destroyed." << std::endl;
}
```

This destructor outputs a message when a `TransformerYY` object is destroyed, indicating which transformer is being destroyed. The `TransformerYY.cpp` file implements the `TransformerYY` class, providing essential functionalities to model a three-phase Y-Y transformer. The constructor initializes the transformer parameters, validates them, and computes the Y parameters necessary for the transformer's operation. The destructor ensures that the resources are properly managed upon object destruction. Overall, this implementation effectively encapsulates the behaviour and parameters of a transformer in a structured manner.

3.2.6 Generator

Introduction In power system analysis, synchronous generators are a fundamental component for energy generation. To model the interaction between the generator and the rest of the power network, admittance parameters (Y-parameters) are often used. The Y-parameters provide a relationship between the voltages and currents at the generator's terminals and can be used in steady-state and dynamic analysis. This report presents the mathematical formulation of the Y-parameters for a synchronous generator, considering key electrical parameters such as field resistance, field inductance, direct-axis reactance, and field time constant.

Generator Model Parameters The generator's behaviour can be described using the following key parameters:

- R_f: Field winding resistance
- L_f: Field winding inductance
- X_d: Direct-axis reactance
- T_f: Field winding time constant

These parameters represent the electrical characteristics of the synchronous machine and are used to derive the generator's impedance and admittance.

Impedance Representation In the frequency domain, the impedance of the generator can be described by its field impedance and direct-axis reactance.

Field Impedance (Z_f) The field winding presents both resistive and inductive characteristics. The impedance of the field is given by:

$$Z_f = R_f + j\omega L_f$$

where:

- j is the imaginary unit.
- ω is the angular frequency.

The inductive component $j\omega L_f$ represents the reactance introduced by the inductance of the field winding at a given frequency.

Direct-Axis Reactance (Z_d) The synchronous reactance along the direct axis of the machine is represented as a purely reactive impedance:

$$Z_d = jX_d$$

where X_d is the direct-axis synchronous reactance.

Field Admittance and Transfer Function The admittance is the reciprocal of impedance, and it provides a direct relationship between the current through and voltage across the generator terminals.

Field Admittance (Y_f) The admittance corresponding to the field impedance is:

$$Y_f = \frac{1}{Z_f} = \frac{1}{R_f + j\omega L_f}$$

This expression defines the field winding's contribution to the overall generator admittance.

Field Transfer Function (H_f) The time dynamics of the generator's field winding are represented by a transfer function based on the field time constant T_f. The field transfer function is expressed as:

$$H_f = \frac{1}{1 + T_f j\omega}$$

This transfer function is crucial for modeling the dynamic behavior of the field winding in the frequency domain. It characterizes the response of the field current to changes in the applied voltage over time.

Y-Parameter Matrix The generator's behavior is captured by a 2-port Y-parameter matrix, which describes the relationship between the currents and voltages at the generator's terminals. The Y-parameter matrix for the generator is given by:

$$\mathbf{Y} = \begin{pmatrix} Y_{11} & Y_{12} \\ Y_{21} & Y_{22} \end{pmatrix}$$

The elements of the matrix are derived as follows:

Self-Admittance Y_{11} The self-admittance Y_{11} is the admittance seen at the generator's field winding, which is given by the field admittance:

$$Y_{11} = Y_f = \frac{1}{R_f + j\omega L_f}$$

This term represents the ability of the field winding to conduct current for a given applied voltage.

Mutual Admittance Y_{12} The mutual admittance Y_{12} represents the coupling between the field winding and the direct-axis impedance. It is derived as:

$$Y_{12} = -\frac{H_f}{Z_d} = -\frac{1}{(T_f j\omega + 1) j X_d}$$

This negative value accounts for the phase relationship between the field and direct-axis circuits.

Reverse Mutual Admittance Y_{21} The reverse mutual admittance, Y_{21}, describes the coupling in the opposite direction, from the direct-axis circuit to the field winding. For simplicity, in many generator models, this term is often approximated as zero:

$$Y_{21} = 0$$

Self-Admittance Y_{22} The self-admittance Y_{22} describes the admittance in the direct-axis circuit. This is represented as:

$$Y_{22} = -\frac{1}{Z_d} = -\frac{1}{j X_d}$$

This term indicates the relationship between the current and voltage in the direct-axis of the generator.

The Y-parameter matrix provides a compact and efficient method for analyzing the steady-state and dynamic behaviour of a synchronous generator. By modelling the field winding impedance, direct-axis reactance, and the field transfer function, we can accurately describe the generator's response to electrical inputs. The self-admittances Y_{11} and Y_{22} provide insight into the field and direct-axis characteristics, while the mutual admittances capture the interaction between the two. This Y-parameter model is essential for understanding the performance of the generator in power system studies.

Code Implementation This report provides a comprehensive explanation of the C++ code implemented for modelling the synchronous generator in the file `Generator.cpp` and the `Generator.h`. The synchronous generator is modelled as an electrical element with specified parameters such as field winding resistance, field winding reactance, direct-axis reactance, and the field winding time constant. For simplicity, we have neglected the governor and excitation system models within the synchronous generator and have also assumed that the mutual and cross-couplings between the phases are relatively small and hence can be neglected while formulating the Y matrices as shown in this report.

Introduction This report explains the structure and functionality of the `generator.h` file, which defines a `Generator` class used in the modeling of electrical generators. The `Generator` class is derived from the base class `Element` and includes parameters specific to the field and direct-axis dynamics of a generator. This header file is an essential part of building a generator's model in a larger electrical system simulation.

Header File Contents Below is the complete `generator.h` file, which defines the generator's interface:

```cpp
// Protection against multiple inclusions
#ifndef GENERATOR_H
#define GENERATOR_H

#include "Element.h" // Base class
#include <array> // Include array for standard containers

class Generator : public Element {
public:
    // Constructor
    Generator(const std::string& symbol, int pins, const std::vector<double>& values);

    // Destructor
    ~Generator() {}

private:
    double R_f = 1.0; // Default value for field resistance
    double L_f = 0.01; // Default value for field inductance
    double X_d = 1.0; // Default value for direct-axis reactance
    double T_f = 0.1; // Default value for field time constant
};

#endif
```

Explanation of Key Sections Let us now break down the key paragraphs of the `generator.h` file:

Header Guards The first section of the file uses header guards to prevent multiple inclusions of the file in a project. This is important to avoid redefinition errors when the header file is included in multiple source files.

```cpp
#ifndef GENERATOR
#define GENERATOR
```

The `#ifndef` and `#define` preprocessor directives ensure that the contents of this file are only included once during compilation. If the symbol `GENERATOR` is already defined, the content is skipped. This is a common practice in C++ header files.

Includes and Dependencies The following includes are used:

```cpp
#include "Element.h"
#include <array>
```

- The `Element.h` file is included because the `Generator` class inherits from the `Element` class, which likely provides common functionalities shared among different electrical components. - The `<array>` library is included to support the use of standard containers, although it is not directly used in this header. It might be used in the corresponding implementation file.

Class Definition The `Generator` class is defined as a derived class from `Element`, indicating that it inherits attributes and methods from the base `Element` class:

```
class Generator : public Element {
```

Public Members The public section of the `Generator` class contains the constructor and destructor.

```
// Constructor
Generator(const std::string& symbol, int pins, const std::vector<double>& values);
```

- The constructor initializes a generator object with a specific `symbol`, a number of `pins`, and a `values` vector that contains the electrical parameters. The symbol likely refers to the label used for the generator in the system, and the pins represent the number of electrical terminals.

```
// Destructor
~Generator() {}
```

- The destructor is defined as a simple empty function. As the class does not manage any dynamic memory (i.e., it does not use pointers), there is no need for custom cleanup logic in the destructor.

Private Members The private section contains four key parameters that describe the electrical characteristics of the generator. These parameters are assigned default values that can be overridden during object initialization.

```
private:
    double R_f = 1.0; // Default value for field resistance
    double L_f = 0.01; // Default value for field inductance
    double X_d = 1.0; // Default value for direct-axis reactance
    double T_f = 0.1; // Default value for field time constant
```

These parameters are: - `R_f`: Field resistance of the generator with a default value of 1.0. - `L_f`: Field inductance, which defaults to 0.01 H. - `X_d`: Direct-axis reactance, with a default value of 1.0. - `T_f`: Time constant of the field winding, with a default value of 0.1 seconds.

These default values are chosen to represent typical behavior for generator components in the absence of specific data, but they can be customized during object construction.

Constructor Details The constructor takes three arguments:

- `symbol`: A string representing the name or label of the generator.
- `pins`: An integer representing the number of pins (or terminals) the generator has in the electrical system.
- `values`: A vector of doubles representing the parameters of the generator, which should correspond to `R_f`, `L_f`, `X_d`, and `T_f`.

When the constructor is invoked, it is expected that the values provided in the `values` vector will overwrite the default parameter values.

The `generator.h` file defines a simple interface for modeling a synchronous generator in a power system simulation. The class inherits from `Element`, which likely provides the basic functionality for electrical elements, and introduces key generator-specific parameters such as field resistance, inductance, direct-axis reactance, and time constant. These parameters are fundamental for calculating the dynamic response of the generator in an electrical network.

Through this file, a user can instantiate a generator model with customizable values or use default settings for typical behaviour. The next step would involve implementing the generator's functionality in the corresponding `generator.cpp` file, where its behaviour would be defined in more detail.

Introduction This report provides a detailed explanation of the `generator.cpp` file. The file defines the constructor and Y-parameter calculations for the `Generator` class. The class is responsible for initializing the generator's parameters and calculating the Y-matrix, which is used in network analysis for admittance modeling.

File Structure Overview The `generator.cpp` file implements the following key components:

- Constructor to initialize the generator with parameters.
- Conversion of physical parameters to symbolic variables using SymEngine.
- Calculation of Y-parameters from the generator's impedance and time constants.
- Population of the Y matrix for network analysis.

Code Explanation

Header Inclusion The file begins by including the corresponding header file for the `Generator` class:

```
// Include the corresponding header file
#include "Generator.h"
```

This line includes the declaration of the `Generator` class and other dependencies defined in `Generator.h`.

Constructor Definition The constructor is defined to initialize the generator object. It takes three parameters: a string symbol, the number of pins, and a vector of values corresponding to the generator's electrical characteristics.

```
// Constructor
Generator::Generator(const std::string& symbol, int pins, const std::vector<double>& values)
    : Element(symbol, pins, pins) {
```

Here, the constructor calls the base class `Element`'s constructor with the symbol and pin count. This inheritance allows the `Generator` to behave as an `Element` in a larger network model.

Parameter Initialization The constructor ensures that the generator has been initialized with exactly four values corresponding to the generator's key parameters.

```
// Validate input size
if (values.size() == 4) {
    R_f = values[0]; // Field resistance
    L_f = values[1]; // Field inductance
    X_d = values[2]; // Direct-axis reactance
    T_f = values[3]; // Field time constant
}
else {
    throw std::invalid_argument("Invalid number of values for generator, must be 4!");
}
```

- `R_f`: Field resistance. - `L_f`: Field inductance. - `X_d`: Direct-axis reactance. - `T_f`: Field time constant. If the size of the `values` vector is not exactly 4, an exception is thrown to ensure proper initialization.

Symbolic Variables The next part of the code defines symbolic variables for the complex frequency, $j\omega$, and converts the parameters into SymEngine symbolic types.

```
// Define symbolic expression for j$\omega$ (complex frequency)
RCP<const Basic> s = mul(j, omega);

// Conversion to SymEngine real double data type
RCP<const Basic> R_f_val = real_double(R_f);
RCP<const Basic> L_f_val = real_double(L_f);
RCP<const Basic> X_d_val = real_double(X_d);
RCP<const Basic> T_f_val = real_double(T_f);
```

- `s`: A symbolic expression representing `j`ω, where `j` is the imaginary unit and ω is the angular frequency. - The parameters `R_f`, `L_f`, `X_d`, and `T_f` are converted to SymEngine's `real_double` type for further symbolic manipulation.

Impedance and Admittance Calculations Next, the field impedance `Z_f`, admittance `Y_f`, and direct-axis impedance `Z_d` are computed.

```
// Field impedance and admittance
RCP<const Basic> Z_f = add(R_f_val, mul(s, L_f_val));
RCP<const Basic> Y_f = div(real_double(1), Z_f);

// Direct-axis impedance
RCP<const Basic> Z_d = mul(I, X_d_val);
```

- `Z_f` is the impedance of the field winding, which consists of the field resistance (`R_f`) and the inductance (`L_f`). - `Y_f` is the admittance, which is the reciprocal of the field impedance. - `Z_d` is the impedance along the generator's direct axis, modeled as a purely reactive impedance using the direct-axis reactance (`X_d`).

Time Constant Function The field time constant dynamics are modeled by `H_f`, a function of the field time constant `T_f` and the complex frequency.

```
// Time constant function
RCP<const Basic> H_f = div(real_double(1), add(mul(T_f_val, s), real_double(1)));
```

- `H_f` describes how the field inductance interacts with the frequency ω and time constant `T_f`.

Y Parameter Calculations The next section defines the Y-parameters based on the field admittance, direct-axis impedance, and time constant function.

```
// Y parameters definition
RCP<const Basic> Y11 = Y_f;
RCP<const Basic> Y12 = neg(div(H_f, Z_d));
RCP<const Basic> Y21 = real_double(0);
RCP<const Basic> Y22 = neg(div(real_double(1), Z_d));
```

- `Y11`: The admittance of the field winding. - `Y12`: The interaction between the time constant function and the direct-axis impedance. - `Y21`: Set to zero, since there is no direct interaction with the other terminal. - `Y22`: The direct-axis admittance, computed as the reciprocal of the direct-axis impedance.

Populating the Y Matrix Finally, the calculated Y-parameters are populated into the generator's Y matrix. The matrix is populated for all the terminals (pins) of the generator. It must be noted here that as the mutual and the cross couplings between the phases are neglected, hence the Y matrix for the three-phase case is defined as a diagonal one as shown here.

```
// Assigning Y matrix values
for (int i = 0; i < pins; i++) {
    Y_matrix.set(i, i, Y11); // Y11
    Y_matrix.set(i, pins + i, Y12); // Y12
    Y_matrix.set(pins + i, i, Y21); // Y21 (symmetrical to Y12)
    Y_matrix.set(pins + i, pins + i, Y22); // Y22
}
```

- The matrix is filled with the calculated Y parameters for each pair of terminals. This matrix is used in network analysis to calculate how the generator interacts with other elements in the system. The `generator.cpp` file implements the constructor for the `Generator` class and handles the symbolic calculations of the Y matrix using SymEngine. The Y matrix is used to describe the generator's admittance in a network, which is crucial for power system analysis. Each section of the code ensures that the generator parameters are properly initialized and calculated, allowing for accurate representation of the generator's behavior in simulations.

3.2.7 Transmission Lines

Long transmission lines are typically modeled using distributed parameters rather than lumped elements due to the significant effect of the line's length on voltage and current distribution. The telegrapher's equations describe the voltage and current along the transmission line as functions of time and space, taking into account resistance (R), inductance (L), capacitance (C), and conductance (G) per unit length. This report aims to derive the Y parameters for a long transmission line, which relates the input voltage and current to the output voltage and current, using the distributed parameter model.

1. Telegrapher's Equations: The telegrapher's equations are partial differential equations that describe the voltage $V(x,t)$ and current $I(x,t)$ at a point x along a transmission line. They are derived based on Kirchhoff's voltage and current laws, combined with the lumped element model for small segments of the line. The telegrapher's equations in the frequency domain are given as:

$$\frac{dV(x)}{dx} = -(R + j\omega L)I(x) \tag{3.2}$$

$$\frac{dI(x)}{dx} = -(G + j\omega C)V(x) \tag{3.3}$$

   where:

   - R is the resistance per unit length (Ω/m),
   - L is the inductance per unit length (H/m),
   - G is the conductance per unit length (S/m),
   - C is the capacitance per unit length (F/m),
   - ω is the angular frequency of the source.

2. Characteristic Impedance and Propagation Constant: From the telegrapher's equations, we can derive two important parameters:

   - **Characteristic Impedance** (Z_0):

$$Z_0 = \sqrt{\frac{R + j\omega L}{G + j\omega C}} \tag{3.4}$$

   - **Propagation Constant** (γ):

$$\gamma = \sqrt{(R + j\omega L)(G + j\omega C)} \tag{3.5}$$

   The characteristic impedance describes the ratio of voltage to current for a wave traveling along the transmission line, while the propagation constant describes how the wave attenuates and changes phase as it propagates along the line.

3. Voltage and Current Distribution Along the Line: The general solutions for the voltage and current along the transmission line are:

$$V(x) = V_+ e^{-\gamma x} + V_- e^{\gamma x} \tag{3.6}$$

$$I(x) = \frac{V_+}{Z_0} e^{-\gamma x} - \frac{V_-}{Z_0} e^{\gamma x} \tag{3.7}$$

   where V_+ and V_- represent the forward and backward traveling voltage waves, respectively. These constants can be determined by applying the boundary conditions at the input and output of the line.

4. Boundary Conditions: At $x = 0$ (the input or source end of the transmission line), the boundary conditions are:

$$V(0) = V_s \tag{3.8}$$

$$I(0) = \frac{V_s - V(0)}{Z_s} \tag{3.9}$$

where V_s is the source voltage and Z_s is the source impedance.

At $x = l$ (the load end of the transmission line), the boundary conditions are:

$$V(l) = V_+ e^{-\gamma l} + V_- e^{\gamma l} \tag{3.10}$$

$$I(l) = \frac{V(l)}{Z_L} \tag{3.11}$$

where Z_L is the load impedance.

5. Computation of Y Parameters: The Y parameters relate the input and output voltages and currents as follows:

$$\begin{bmatrix} I_1 \\ I_2 \end{bmatrix} = \begin{bmatrix} Y_{11} & Y_{12} \\ Y_{21} & Y_{22} \end{bmatrix} \begin{bmatrix} V_1 \\ V_2 \end{bmatrix}$$

We compute the Y parameters using the voltage and current expressions at both ends of the transmission line ($x = 0$ and $x = l$):

$$I_1 = \frac{V_+}{Z_0} - \frac{V_-}{Z_0}$$
$$I_2 = \frac{V_+}{Z_0} e^{-\gamma l} - \frac{V_-}{Z_0} e^{\gamma l}$$

From these expressions, we derive the following Y parameters:

$$Y_{11} = \frac{1}{Z_0} \frac{\sinh(\gamma l)}{\cosh(\gamma l)}$$
$$Y_{12} = \frac{-1}{Z_0} \frac{1}{\cosh(\gamma l)}$$
$$Y_{21} = Y_{12}$$
$$Y_{22} = Y_{11}$$

These Y parameters describe the transmission line's behavior regarding its input and output voltages and currents.

In this report, we derived the Y parameters for a long transmission line using the telegrapher's equations. We first computed the characteristic impedance and propagation constant, which describe how voltage and current waves propagate along the line. Using the boundary conditions at the source and load ends, we then solved for the travelling wave coefficients. We computed the Y parameters, which are crucial for analyzing transmission lines in networked power systems.

Implementation in Code This report provides a comprehensive explanation of the C++ code implemented for modelling a transmission line in the file `transmissionline.cpp` and the `transmissionline.h`. The transmission line is modelled as a long line incorporating series resistance and series inductance per unit length and the shunt conductance and the susceptance per unit length.

Introduction The 'TransmissionLine.h' file defines a class for modeling transmission lines in an electrical circuit simulation framework. It extends from a base class called 'Element', inheriting its properties and methods. This class encapsulates the necessary parameters for analyzing transmission lines, such as resistance, inductance, conductance, capacitance, and line length.

Header Guard The header guard is implemented to prevent multiple inclusions of the same header file, which could lead to redefinition errors.

```
// Header guard
#ifndef TRANSMISSIONLINE_H
#define TRANSMISSIONLINE_H
```

Includes The class includes the 'Element.h' file, which is the base class for the 'TransmissionLine'.

```
#include "Element.h"
```

Class Definition The 'TransmissionLine' class is defined with public and private access specifiers.

```
class TransmissionLine : public Element {
public:
    // Parameterized constructor that calls the base class constructor
    TransmissionLine(const std::string& symbol, int pins, const std::vector<double>&);

    ~TransmissionLine() {}
```

Constructor The constructor initializes an instance of the 'TransmissionLine' class. It takes three parameters:

- 'symbol': A string representing the symbol of the transmission line.
- 'pins': An integer representing the number of pins for the element.
- 'values': A vector of doubles that can hold various parameters for the transmission line.

Destructor The destructor is defined as a trivial operation, indicating that no special cleanup is required when the object is destroyed.

```
~TransmissionLine() {}
```

Private Member Variables The following private member variables represent the transmission line parameters, initialized with default values:

- `R_tl = 0.01`: Represents the resistance per unit length of the transmission line in ohms per meter (Ω/m).
- `L_tl = 2.5e-7`: Represents the inductance per unit length of the transmission line in henries per meter (H/m).
- `G_tl = 1e-9`: Represents the conductance per unit length of the transmission line in siemens per meter (S/m).

- `C_tl = 1e-11`: Represents the capacitance per unit length of the transmission line in farads per meter (F/m).
- `length = 1000`: Represents the length of the transmission line in meters (m).

The member variables are defined as follows:

```
private:
    double R_tl = 0.01; // Resistance per unit length (ohms/m)
    double L_tl = 2.5e-7; // Inductance per unit length (H/m)
    double G_tl = 1e-9; // Conductance per unit length (S/m)
    double C_tl = 1e-11; // Capacitance per unit length (F/m)
    double length = 1000; // Length of the transmission line (m)
```

The 'TransmissionLine.h' file provides a clear and structured way to define a transmission line in the context of electrical simulations. By inheriting from the 'Element' class, it maintains consistency within the overall framework, allowing for straightforward integration with other circuit elements. The parameters defined in this class can be utilized for further computations, such as calculating admittance or impedance in circuit analysis.

Furthermore, the 'TransmissionLine.cpp' file contains the implementation of the 'TransmissionLine' class, which models the behaviour of transmission lines in electrical circuit simulations. The class extends the 'Element' base class, initializing key parameters related to the transmission line, computing its admittance parameters, and storing them in a matrix format.

Code Explanation The implementation begins with including the 'TransmissionLine.h' header file, which declares the 'TransmissionLine' class.

```
#include "TransmissionLine.h"
```

Constructor Implementation The constructor of the 'TransmissionLine' class initializes its parameters based on the provided values. It validates the size of the input vector to ensure it contains the correct number of parameters.

Constructor Definition The constructor is defined as follows:

```
TransmissionLine::TransmissionLine(const std::string& symbol, int pins, const std::vector<double>& values)
    : Element(symbol, pins, pins) {
```

The constructor takes three parameters:

- `symbol`: A string representing the symbol of the transmission line.
- `pins`: An integer indicating the number of pins.
- `values`: A vector containing five double values representing the parameters of the transmission line.

Parameter Initialization and Validation The constructor checks the size of the 'values' vector. If the size is not equal to 5, it throws an exception. Otherwise, it assigns the values to the respective member variables.

```
    if (values.size() == 5) {
        R_tl = values[0]; // Resistance per unit length (ohms/m)
        L_tl = values[1]; // Inductance per unit length (H/m)
        G_tl = values[2]; // Conductance per unit length (S/m)
        C_tl = values[3]; // Capacitance per unit length (F/m)
        length = values[4]; // Length of the transmission line (m)
    } else {
        throw std::invalid_argument("Invalid number of values, must be 5 (resistance, inductance, conductance,
            capacitance and length)!");
    }
```

Parameter Computation The constructor continues by defining complex parameters necessary for calculating the admittance matrix.

Parameter Conversion The resistance, inductance, conductance, and capacitance values are converted to a specific type using 'real_double', which presumably returns a representation suitable for further calculations.

```
    RCP<const Basic> R_val = real_double(R_tl);
    RCP<const Basic> L_val = real_double(L_tl);
    RCP<const Basic> G_val = real_double(G_tl);
    RCP<const Basic> C_val = real_double(C_tl);
```

Complex Frequency Components Next, the constructor computes intermediate values based on the angular frequency, inductance, and capacitance:

```
    RCP<const Basic> omega_L = mul(omega, L_val);
    RCP<const Basic> omega_C = mul(omega, C_val);
    RCP<const Basic> R_plus_i_omega_L = add(R_val, mul(I, omega_L));
    RCP<const Basic> G_plus_i_omega_C = add(G_val, mul(I, omega_C));
```

Impedance and Propagation Constants The characteristic impedance Z_0 and propagation constant γ are computed using the previously defined parameters.

```
    RCP<const Basic> Z0 = sqrt(div(R_plus_i_omega_L, G_plus_i_omega_C));
    RCP<const Basic> gamma = sqrt(mul(R_plus_i_omega_L, G_plus_i_omega_C));
```

Subsequently, the length-dependent parameters are computed:

```
    RCP<const Basic> gamma_l = mul(gamma, real_double(length));
    RCP<const Basic> cosh_gamma_l = cosh(gamma_l);
    RCP<const Basic> sinh_gamma_l = sinh(gamma_l);
    RCP<const Basic> Z0_sinh_gamma_l = mul(Z0, sinh_gamma_l);
```

Y-Parameter Calculation The admittance parameters Y_{11}, Y_{12}, Y_{21}, and Y_{22} are calculated based on the previously computed values.

```
    RCP<const Basic> Y11 = div(neg(cosh_gamma_l), Z0_sinh_gamma_l);
    RCP<const Basic> Y12 = div(real_double(1), Z0_sinh_gamma_l);
    RCP<const Basic> Y21 = div(real_double(1), Z0_sinh_gamma_l);
    RCP<const Basic> Y22 = div(cosh_gamma_l, Z0_sinh_gamma_l);
```

Storing Y-Parameters in Matrix The computed Y-parameters are stored in the 'Y_matrix'. The loop iterates through the number of pins and assigns the calculated values:

```
    for (int i = 0; i < pins; i++) {
        Y_matrix.set(i, i, Y11); // Y11
        Y_matrix.set(i, pins + i, Y12); // Y12
        Y_matrix.set(pins + i, i, Y21); // Y21 (symmetrical to Y12)
        Y_matrix.set(pins + i, pins + i, Y22); // Y22
    }
```

The 'TransmissionLine.cpp' file implements the logic necessary to model transmission lines in electrical circuits. By computing the admittance parameters based on the physical characteristics of the line, the class provides essential data for further analysis and simulation. The design incorporates error handling for invalid input, ensuring robustness in the construction of transmission line objects.

3.3 Equivalent Impedance of the Subnetwork

The HVDC system induces harmonics in the grid. These harmonics entail serious stability issues and are a real source of problems. Before going into system modeling to study the harmonic effect, a basic overview of the nature of the generated harmonics is important—a brief but detailed overview of the nature of harmonics or the multiples of generated harmonics.

3.3.1 Harmonic Resonance Modal Analysis Approach

Y parameters can determine the impedance between input and output pins. It uses the harmonic resonance modal analysis approach [1, 2]. The input impedance can be calculated using the following steps. First, the matrix $\mathbf{M}$ is constructed as a zero quadratic matrix with the size $N \times N$. The entries that belong to the currents of the input and output nodes, denoted as k_i, $i \in \{1, \ldots, N\}$, are equaled with $\mathbf{M}\langle k_i, k_i \rangle = 1$.

$$\begin{bmatrix} Y_{1,1} & Y_{1,2} & \cdots & Y_{1,k_i} & \cdots & Y_{1,N} \\ Y_{2,1} & Y_{2,2} & \cdots & Y_{2,k_i} & \cdots & Y_{2,N} \\ \vdots & \vdots & \ddots & \vdots & \ddots & \vdots \\ Y_{k_i,1} & Y_{k_i,2} & \cdots & Y_{k_i,k_i} & \vdots & Y_{k_i,N} \\ \vdots & \vdots & \ddots & \vdots & \ddots & \vdots \\ Y_{N,1} & Y_{N,2} & \cdots & Y_{N,k_i} & \cdots & Y_{N,N} \end{bmatrix} \begin{bmatrix} V_1 \\ V_2 \\ \vdots \\ V_k \\ \vdots \\ V_N \end{bmatrix} = \mathbf{M} \begin{bmatrix} I_1 \\ I_2 \\ \vdots \\ I_k \\ \vdots \\ I_N \end{bmatrix}, \tag{3.12}$$

The inverse of the matrix $\mathbf{Y}$, even in the case of singularity, can be determined as [2]:

$$\mathbf{Z} = \mathbf{T}\,\mathbf{\Lambda}\,\mathbf{T}^{-1},$$

where $\mathbf{T}$ presents column eigenvectors and $\mathbf{\Lambda}$ diagonal complex eigenvalues of the matrix $\mathbf{Y}$.

The last step is to determine the impedances between input and output pins as:

$$\mathbf{Z}_{eq} = \mathbf{Z}\,\mathbf{M}$$

and to pick only the values on the positions corresponding to the input and output nodes.

To get the impedance "visible" from the input nodes, it is necessary to perform Kron elimination of the output nodes as the last step [3]. The so-called Kron reduction [3], is often used for reducing grounded conducting layers of the overhead lines and cables. It will be formulated only for the Y parameters, but it can be applied in the same way for Z parameters.

The Y matrix is divided into four parts: the matrix part with the superscript 11 should be kept, 22 should be eliminated and 12 and 21 are the interconnections.

$$\mathbf{Y} = \begin{bmatrix} \mathbf{Y}_{11} & \mathbf{Y}_{12} \\ \mathbf{Y}_{21} & \mathbf{Y}_{22} \end{bmatrix}$$

The new Y parameters are then given by:

$$\widetilde{\mathbf{Y}} = \mathbf{Y}_{11} - \mathbf{Y}_{12}\,\mathbf{Y}_{22}^{-1}\,\mathbf{Y}_{21}.$$

3.3.2 Adjusted Modified Nodal Analysis Approach

The solving procedure can be successfully applied with the following steps:

1. Determine the input and output of the circuit where you want to determine the equivalent impedance, see Fig. 3.5.
2. Assign input and output currents as depicted in Fig. 3.6.
3. Build Modified Nodal Analysis (MNA) [4] by adding all components except the ones on the direct connection to the input and output pins. Those components are considered as non-existent as seen in Fig. 3.6. Furthermore, the following sorting of variables should be applied: $[V_i, I_i, V_o, I_o, V]$, where

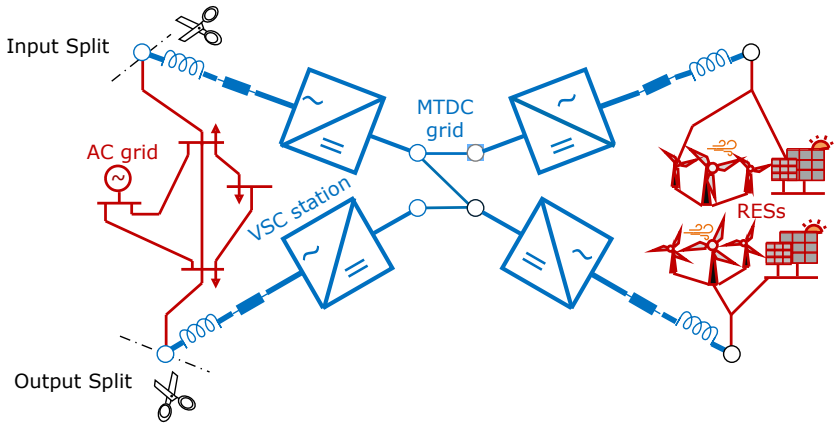

Figure 3.5: Circuit cut.

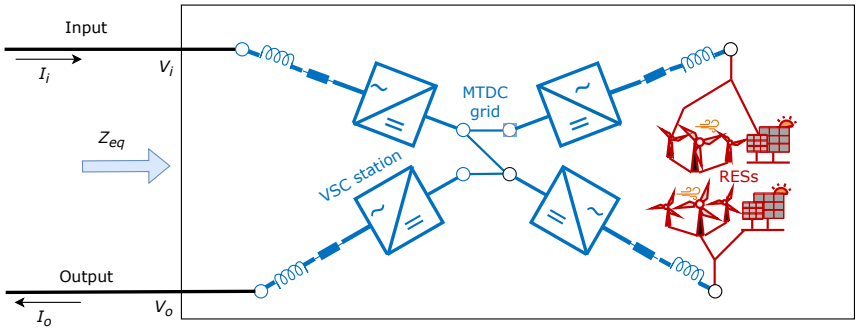

Figure 3.6: Equivalent impedance determination.

V_i and I_i present voltages and currents at the input like depicted in Fig. 3.6, V_o and I_o are output voltages and currents, and V are other voltage buses/nodes that are not input and output ones.

There are no equations written for the ground bus, as standardly defined in the MNA approach [4].

4. The formulated equations will be as follows:

$$\underbrace{\begin{bmatrix} Y_{ii} & \text{diag}\{-1\} & Y_{io} & 0 & Y_{iv} \\ \text{diag}\{1\} & 0 & 0 & 0 & 0 \\ Y_{oi} & 0 & Y_{oo} & \text{diag}\{1\} & Y_{ov} \\ 0 & 0 & \text{diag}\{1\} & 0 & 0 \\ Y_{vi} & 0 & Y_{vo} & 0 & Y_{vv} \end{bmatrix}}_{M} \underbrace{\begin{bmatrix} V_i \\ I_i \\ V_o \\ I_o \\ V \end{bmatrix}}_{x} = \underbrace{\begin{bmatrix} V_1 \\ 0 \\ V_2 \\ 0 \\ 0 \end{bmatrix}}_{z}, \tag{3.13}$$

where Y_{xy} are Y parameters for the specified block of nodes x and y. Values V_1 and V_2 are symbolic vectors of values assigned to input and output nodes. With 0 is denoted zero matrix. For simplicity, the dimensions are skipped in the previous equation.

5. The previous equation (3.13) is solved using the reduced row echelon form [5] on the concatenated matrices M and z. This will solve the system of equations by enforcing the values for variables x to diagonal 1 values, and on the position of z column would be the expected solution.

6. By taking values for V_i and I_i and dividing them, we obtain the equivalent impedance per phase.

References

[1] H. A. Brantsæter, "Harmonic resonance mode analysis and application for offshore wind power plants," Master's thesis, NTNU, 2015.

[2] W. Xu, Z. Huang, Y. Cui, and H. Wang, "Harmonic resonance mode analysis," *IEEE Transactions on Power Delivery*, vol. 20, no. 2, pp. 1182–1190, 2005.

[3] F. Dorfler and F. Bullo, "Kron reduction of graphs with applications to electrical networks," *IEEE Transactions on Circuits and Systems I: Regular Papers*, vol. 60, no. 1, pp. 150–163, 2012.

[4] C.-W. Ho, A. Ruehli, and P. Brennan, "The modified nodal approach to network analysis," *IEEE Transactions on Circuits and Systems*, vol. 22, no. 6, pp. 504–509, 1975.

[5] J. Bird, *Electrical circuit theory and technology*. Routledge, 2017.